OS Report 2026

การเขียนโปรแกรมแบบขนาน และการแสดงภาพ Memory Map

ข้อ 1 การคูณเมทริกซ์แบบขนานด้วย OpenMP
ข้อ 2 เครื่องมือแสดงภาพ memory map ของโปรเซสบน Linux

จัดทำโดย

ธยศ ขันทะยศ

รหัสนิสิต 6610505420

รายงานนี้เป็นส่วนหนึ่งของรายวิชา 01204332 ระบบปฏิบัติการ

27 ตุลาคม 2569

สารบัญ

บทนำ **1**

ส่วนที่ 1 การคูณเมทริกซ์แบบขนานด้วย OpenMP **1**

1.1 สภาพแวดล้อมการทดลอง 1

1.2 อัลกอริทึม 2

1.3 วิธีการวัด 4

1.4 ผลการทดลองและการวิเคราะห์ 5

1.4.1 (a) จาก 1 thread เป็นหลาย thread 5

1.4.2 (b) จากขนาดเล็กไปขนาดใหญ่ 7

1.4.3 (c) ค่าตั้งของโปรแกรม 9

1.4.4 (d) ค่าตั้งของระบบปฏิบัติการและ OpenMP runtime 12

1.4.5 (e) ความเร็วและหน่วยความจำ 16

1.5 การวิเคราะห์ด้วย Amdahl's law 17

1.6 ข้อจำกัดของการทดลอง 20

ส่วนที่ 2 การแสดงภาพ Memory Map ของโปรเซส **21**

2.1 เป้าหมายและข้อมูลขาเข้า 21

2.2 การออกแบบและการพัฒนา 21

2.3 ความสามารถและวิธีใช้งาน 22

2.4 กรณีศึกษา: โปรแกรม demo 23

2.5 ข้อสังเกตและการวิเคราะห์ 26

2.6 ข้อจำกัด 28

ส่วนที่ 3 การใช้ปัญญาประดิษฐ์ (AI) **28**

สรุป **28**

เอกสารอ้างอิง **30**

บทนำ

รายงานฉบับนี้แบ่งเป็นสองส่วนตามโจทย์ ส่วนที่ 1 ศึกษาการคูณเมทริกซ์จัตุรัส C = A × B ขนาด n × n แบบขนานด้วย OpenMP [1] บนเครื่อง 8 คอร์ โดยวัดผลของจำนวน thread ขนาดปัญหา ค่าตั้งของโปรแกรม และค่าตั้งของระบบปฏิบัติการ ทั้งในด้านเวลาและหน่วยความจำ แล้ววิเคราะห์สัดส่วนของส่วนที่ทำงานขนานกับส่วนที่ทำงานแบบลำดับ (serial) ด้วย Amdahl's law [2] ส่วนที่ 2 พัฒนาเครื่องมือแสดงภาพ memory map ของโปรเซสบน Linux จากข้อมูลที่ kernel เปิดเผยไว้ใน /proc แล้วใช้เครื่องมือนี้ติดตามการเปลี่ยนแปลงของ address space ของโปรแกรมตัวอย่างทีละขั้น

ซอร์สโค้ดทั้งหมดอยู่ในโฟลเดอร์ `os_report_6610505420` แบ่งเป็น `1_parallel_6610505420` และ `2_mmap_6610505420` ผลการทดลองในส่วนที่ 1 มาจากโฟลเดอร์ `results/` ได้แก่ไฟล์ CSV ที่ `bench.py` บันทึก ตารางสรุป `summary.md` และกราฟใน `plots/` ที่ `plot.py` สร้าง ส่วนตัวเลขและภาพในส่วนที่ 2 มาจากไฟล์ `demo_timeline.html` ที่ `mmap.py` บันทึกไว้ขณะรันโปรแกรม `demo` บน WSL2 ตัวเลขใดที่คำนวณต่อจากผลการวัดจะแสดงวิธีคำนวณกำกับไว้ทุกครั้ง

ส่วนที่ 1 การคูณเมทริกซ์แบบขนานด้วย OpenMP

1.1 สภาพแวดล้อมการทดลอง

ตารางที่ 1 สภาพแวดล้อมของการทดลอง (จาก `results/system.txt` และข้อมูลจำเพาะของผู้ผลิต)

CPU	AMD Ryzen 7 9700X (สถาปัตยกรรม Zen 5) 8 คอร์ 16 hardware thread, base clock 3.8 GHz, boost สูงสุด 5.5 GHz, TDP 65 W [3]
CPU ที่ VM เห็น	8 CPU, 1 thread ต่อคอร์, Hypervisor vendor: Microsoft (lscpu)
L1 data cache	48 KiB ต่อคอร์ (รวม 384 KiB), 12-way, cache line 64 ไบต์ [4]
L2 cache	1 MiB ต่อคอร์ (รวม 8 MiB), 16-way [4]
L3 cache	32 MiB ก้อนเดียวใช้ร่วมกันทั้ง 8 คอร์, NUMA node เดียว
TLB ข้อมูล	L1 DTLB 96 entry, L2 DTLB 4,096 entry (ใช้กับหน้า 4 KiB และ 2 MiB) [4]
SIMD	AVX-512 บน datapath กว้าง 512 บิต และ FMA [4]
หน่วยความจำ	RAM 15 GiB และ swap 4 GiB ที่ WSL2 จัดให้ VM
ระบบปฏิบัติการ	Ubuntu 24.04 ใน WSL2 บน Windows 11, kernel 6.6.87.2-microsoft-standard-WSL2, glibc 2.39, Python 3.12.3 [5]
Compiler	gcc 13.3.0 กับ OpenMP runtime ของ GNU (libgomp) คอมไพล์ด้วย `-std=gnu11 -O3 -march=native -fopenmp` [6]
Huge pages	ค่าของระบบ transparent_hugepage/enabled = madvise และ defrag = madvise; โปรแกรมขอหรือไม่ขอ huge pages ผ่าน `madvise()` เฉพาะช่วงหน่วยความจำของเมทริกซ์ ไม่แก้ค่าของระบบ [7]

เหตุที่ต้องระบุขนาดและ associativity ของ cache ทุกระดับ คือผลการทดลองหลายข้ออธิบายได้ด้วยการเทียบขนาดข้อมูลที่โปรแกรมต้องใช้ซ้ำ (working set) กับความจุของ cache แต่ละระดับ ได้แก่ ความเร็วของ ikj ที่ลดลงมากเมื่อ n โต ขนาด tile ที่ดีที่สุดของ blocked และความช้าผิดปกติเมื่อ n เป็นกำลังของสอง ส่วนจำนวน

entry ของ TLB ใช้อธิบายผลของ huge pages และค่า THP ของระบบที่เป็น madvise หมายความว่าการรันแบบ default ไม่ได้ใช้ huge pages เลย ซึ่งยืนยันได้จาก AnonHugePages = 0 ในทุกการรันที่ไม่ได้ขอ

การทดลองรันใน WSL2 ซึ่งเป็น virtual machine ขนาดเบาบน Hyper-V [5] จึงมีผลต่อการวัดอย่างน้อยสามด้าน ด้านแรก Linux ใน VM เห็น CPU เป็น virtual CPU (vCPU) 8 ตัว ขณะที่ชิปจริงมี 16 hardware thread และ Windows host ยังรันโปรแกรมอื่นอยู่บนคอร์ชุดเดียวกัน vCPU จึงอาจถูก host พักชั่วคราวโดยที่ scheduler ของ Linux ไม่รู้ ด้านที่สอง การแปลที่อยู่ใน VM ต้องอาศัยตารางหน้า (page table) สองชั้นคือของ guest และของ host ทำให้ TLB miss แต่ละครั้งมีราคาสูงกว่าบนเครื่องจริง [8] และ WSL2 คืนหน่วยความจำที่ VM ไม่ใช้ให้ Windows ได้ การแตะหน่วยความจำก้อนใหญ่ครั้งแรกจึงอาจต้องรอให้ host จัดหน้าจริงให้ก่อน ด้านที่สาม ความถี่ของ CPU ไม่คงที่ ชิปเร่งความถี่ได้ถึง 5.5 GHz เมื่อทำงานไม่กี่คอร์ แต่ภายใต้ TDP 65 W ความถี่ขณะที่ทุกคอร์ทำงานพร้อมกันจะต่ำกว่านั้น [3] ทั้งสามด้านนี้มีร่องรอยชัดเจนในข้อมูลดังจะเห็นในหัวข้อ 1.4

1.2 อัลกอริทึม

ทุกอัลกอริทึมคำนวณ $C = A \times B$ โดยเก็บเมทริกซ์ชนิด double แบบ row-major คือ A[i][k] อยู่ที่ตำแหน่ง $i \cdot n + k$ ของอาร์เรย์ งานคำนวณเท่ากันทุกแบบ คือการคูณแล้วบวก n^3 ครั้ง หรือ $2n^3$ FLOP และใช้หน่วยความจำของเมทริกซ์ $3n^2 \cdot 8$ ไบต์ (transpose ใช้ $4n^2 \cdot 8$ ไบต์เพราะต้องเก็บ B^T เพิ่ม) ความแตกต่างอยู่ที่ลำดับการเข้าถึงหน่วยความจำ cache โหลดข้อมูลเป็นก้อนละ 64 ไบต์ (cache line) หรือ double 8 ตัว การอ่านข้อมูลที่อยู่ติดกัน (stride 1) จึงได้ใช้ข้อมูลครบทั้ง 8 ตัวต่อการโหลดหนึ่งครั้ง ส่วนการอ่านข้ามทีละ n ตัว (stride n) ได้ใช้เพียง 1 ใน 8 ตัวก่อนที่ line นั้นจะถูกไล่ออก [9]

ตารางที่ 2 เปรียบเทียบสี่อัลกอริทึมใน `parallel.c`

อัลกอริทึม	loop ในสุด	การเข้าถึง B	loop ที่แบ่งให้ thread	สิ่งที่แต่ละ thread เขียน
naive (i-j-k)	s += A[i][k]·B[k][j] วนตาม k	ตามคอลัมน์ stride n	i	แถว i ของ C
ikj	C[i][j] += a·B[k][j] วนตาม j	ตามแถว stride 1	i	แถว i ของ C
transpose	ผลคูณจุดของแถว i ของ A กับแถว j ของ B^T	B^T ตามแถว stride 1	j (ตอนสลับ), i (ตอนคูณ)	แถวของ B^T และแถว i ของ C
blocked	ลำดับ i-k-j ภายใน tile ขนาด bs × bs	ตามแถวภายใน tile และใช้ tile ซ้ำ bs ครั้ง	(ib, jb) ด้วย collapse(2)	tile (ib, jb) ของ C

naive เป็นสูตรตามตำรา loop ในสุดวนตาม k และอ่าน B[k][j] ซึ่งห่างกันครั้งละ $n \cdot 8$ ไบต์ ทุกการอ่านจึงแตะ cache line ใหม่ และ line เหล่านั้นจะได้ใช้อีกครั้ง (กับคอลัมน์ j+1 ถึง j+7 ที่อยู่ line เดียวกัน) ก็ต่อเมื่อทั้งคอลัมน์ n line ยังอยู่ใน cache นอกจากนี้ loop ในสุดเป็นการสะสมผลบวกลงตัวแปร s ตัวเดียว compiler จะแปลงเป็นคำสั่ง SIMD ไม่ได้หากไม่อนุญาตให้สลับลำดับการบวกเลขทศนิยม ซึ่งค่าเริ่มต้นของ gcc ไม่อนุญาต

ikj สลับลำดับ loop เป็น i-k-j สำหรับแต่ละ k นำ a = A[i][k] ไปคูณทั้งแถว k ของ B แล้วบวกสะสมเข้าแถว i ของ C loop ในสุดอ่าน B และเขียน C แบบ stride 1 และแต่ละรอบของ j ไม่ขึ้นต่อกัน compiler จึงใช้ SIMD (AVX-512 ทำงานกับ double 8 ตัวต่อคำสั่ง) และ FMA ได้เต็มที่ แถว i ของ C ($n \cdot 8$ ไบต์) อยู่ใน L1/L2 ตลอด แต่ทั้งเมทริกซ์ B ถูกอ่านใหม่ทุกครั้งที่เปลี่ยน i การใช้ B ซ้ำจาก cache จึงเกิดได้ก็ต่อเมื่อ B ทั้งก้อน ($n^2 \cdot 8$ ไบต์) จุลงใน cache

transpose คัดลอก B เป็น B^T ก่อน (งาน $O(n^2)$ และหน่วยความจำเพิ่ม $n^2 \cdot 8$ ไบต์) จากนั้น C[i][j] คือผลคูณจุดของสองแถวที่อ่านแบบ stride 1 ทั้งคู่ loop ในสุดใช้ `#pragma omp simd reduction(+:s)` เพื่ออนุญาตให้ compiler แยกผลบวกเป็นหลายช่องของ vector แล้วรวมตอนท้าย สังเกตว่าแถว i ของ A ถูกใช้ซ้ำกับทุก j แต่ B^T ทั้งก้อนยังถูกอ่านใหม่ทุกครั้งที่เปลี่ยน i เช่นเดียวกับ ikj

blocked แบ่ง C เป็น tile ขนาด bs × bs (ค่าเริ่มต้น bs = 64) และแบ่งมิติ k เป็นช่วงละ bs ภายใน tile ใช้ลำดับ i-k-j แบบเดียวกับ ikj จุดสำคัญคือ tile ของ B ($bs^2 \cdot 8$ ไบต์ เช่น 32 KiB เมื่อ bs = 64) ถูกใช้ซ้ำกับทั้ง bs แถวของ tile C ขณะที่ยังอยู่ใน cache ข้อมูลที่ต้องดึงจากหน่วยความจำระดับที่ช้าต่อหนึ่ง FLOP จึงลดลงประมาณ bs เท่า เมื่อเทียบกับ ikj [10], [11]

```
/* blocked (tiled) i-k-j: แต่ละ tile (ib, jb) ของ C เป็นของ thread เดียว จึงไม่ต้อง synchronise */
const int nb = (n + bs - 1) / bs;
#pragma omp parallel for collapse(2) schedule(runtime)
for (int ib = 0; ib < nb; ib++)
  for (int jb = 0; jb < nb; jb++) {
    ... ล้าง tile ของ C ...
    for (int k0 = 0; k0 < n; k0 += bs)            /* ช่วงของ k ทีละ bs */
      for (int i = i0; i < i1; i++)
        for (int k = k0; k < k1; k++) {
          const double a = A[i*n + k];
          for (int j = j0; j < j1; j++)              /* stride 1: SIMD + FMA */
            C[i*n + j] += a * B[k*n + j];
        }
  }
```

การขนานทำด้วย `#pragma omp parallel for` ที่ loop นอกสุด สำหรับ naive, ikj และ transpose loop i ถูกแบ่งเป็นช่วงให้แต่ละ thread แต่ละ thread จึงเขียนเฉพาะแถวของ C ที่ตัวเองได้รับ ส่วน A และ B ถูกอ่านอย่างเดียว ไม่มีตำแหน่งใดที่สอง thread เขียนพร้อมกัน จึงไม่มี race condition และไม่ต้องใช้ lock หรือ atomic สำหรับ blocked คำสั่ง `collapse(2)` รวม loop ib และ jb เป็นงานย่อยอิสระ nb^2 ชิ้น ($nb = \lceil n/bs \rceil$) แต่ละชิ้นเป็นเจ้าของ tile หนึ่งของ C และวน k ทั้งหมดอยู่ภายใน thread เดียว เหตุที่ไม่แบ่งมิติ k ให้หลาย thread เพราะจะทำให้หลาย thread บวกเข้าช่องเดียวกันของ C ซึ่งต้องมี reduction หรือ atomic เพิ่ม ใน transpose ขั้นสลับ B แต่ละ thread เขียนคนละแถวของ B^T และ barrier โดยนัยที่ท้าย parallel for รับประกันว่า B^T เสร็จครบก่อนเริ่มคูณ ทุก loop ใช้ `schedule(runtime)` เพื่อให้เลือกชนิดของ schedule และขนาด chunk ได้จาก command line (ค่าเริ่มต้นคือ static ซึ่งแบ่ง loop เป็นช่วงต่อเนื่องเท่าๆ กัน) [1]

จำนวนงานย่อยของ blocked มีผลมากเมื่อ n เล็ก เมื่อ n = 100 และ bs = 64 จะได้ nb = 2 คือมีเพียง 4 tile ใช้งาน thread ได้แค่ 4 ตัวแม้จะสั่ง 8 thread เมื่อ n = 400 มี 7 × 7 = 49 tile (เฉลี่ย 6.1 tile ต่อ thread เมื่อ p = 8) และเมื่อ n = 3200 มี 50 × 50 = 2,500 tile ซึ่งมากพอให้แบ่งงานได้สมดุล

การตรวจความถูกต้อง ค่าของ A และ B สร้างจากฟังก์ชัน hash (splitmix64) ของตำแหน่งสมาชิก จึงได้เมทริกซ์ชุดเดิมทุกครั้ง ไม่ว่าจะเติมค่าแบบลำดับหรือแบบขนาน หลังคูณเสร็จ โปรแกรมตรวจผลด้วย Freivalds' algorithm [12] คือสุ่มเวกเตอร์ x แล้วเทียบ A(Bx) กับ Cx ซึ่งใช้งานเพียง $O(n^2)$ จึงตรวจได้ทุกขนาด และถือว่าผ่านเมื่อ relative error น้อยกว่า 10^{-9} ผลคือการรันในชุด scaling ผ่านการตรวจครบ 1,170 จาก 1,170 ครั้ง ส่วนตัวเลือก `--verify full` เทียบทุกสมาชิกกับผลของ ikj แบบ thread เดียว ($O(n^3)$) ใช้กับ n เล็ก

คอลัมน์ checksum (ผลรวมทุกสมาชิกของ C) ใน scaling.csv ให้ข้อมูลเพิ่มเติมสองข้อ ข้อแรก ทุกอัลกอริทึมให้ checksum เดิมทุกบิตไม่ว่าจะใช้กี่ thread เพราะการแบ่งงานเปลี่ยนเพียงว่า thread ใดคำนวณสมาชิกใด ไม่ได้เปลี่ยนลำดับการบวกภายในสมาชิก ข้อสอง ikj กับ blocked ให้ checksum ตรงกันทุกบิต (เช่น

3.6136860518863934 ที่ n = 100) เพราะทั้งคู่บวกผลคูณเรียงตาม k จาก 0 ถึง n – 1 ด้วยคำสั่ง FMA ชุดเดียวกัน ขณะที่ naive (...8867522) และ transpose (...8871314) ต่างไปในหลักที่ 13–14 การคอมไพล์ parallel.c ด้วย gcc 13.3 รุ่นเดียวกัน (กับ `-march=znver4` ซึ่งใกล้เคียง) แสดงว่า loop ในสุดของ naive ใช้ vmulsd ตามด้วย vaddsd ซึ่งปัดเศษสองครั้ง แทนที่จะใช้ FMA ที่ปัดเศษครั้งเดียวเหมือน ikj และ blocked ส่วน transpose ใช้ omp simd reduction ที่แยกผลบวกเป็นหลายผลรวมย่อย เนื่องจากการบวกเลขทศนิยมไม่มีสมบัติการเปลี่ยนกลุ่ม (non-associative) ผลจึงต่างกันเล็กน้อย แต่ทุกแบบผ่านเกณฑ์ 10^{-9}

1.3 วิธีการวัด

โปรแกรมจับเวลาแต่ละเฟสแยกกันด้วย `omp_get_wtime()` ได้แก่ *alloc* (จองหน่วยความจำ, serial) *spawn* (parallel region แรก ซึ่ง runtime สร้างทีม thread) *init* (เติมค่า A และ B ซึ่งเป็น serial โดยค่าเริ่มต้น หรือขนานเมื่อใช้ `-p`) *compute* (การคูณ ถ้าคูณซ้ำ r รอบจะใช้รอบที่เร็วที่สุด) และ *checksum* (serial) เวลารวมของโปรแกรมคือผลบวกของห้าเฟสนี้ ส่วนเวลาตรวจผลไม่นับรวม ความเร็วรายงานเป็น GFLOP/s = $2n^3$ / เวลา compute การแยกเฟสแบบนี้ทำให้วัดส่วน serial ได้โดยตรงสำหรับการวิเคราะห์ในหัวข้อ 1.5

bench.py รันแต่ละค่าตั้งเป็นโปรเซสแยกกันทีละโปรเซส ซ้ำ 3 รอบ (5 รอบสำหรับชุด scaling และ parinit) โดยให้รอบซ้ำเป็น loop นอกสุด เพื่อให้สัญญาณรบกวนชั่วคราวจากเครื่องกระทบเพียงหนึ่งตัวอย่างของหลายค่าตั้ง แทนที่จะกระทบทุกตัวอย่างของค่าตั้งเดียว จากนั้น plot.py ใช้ค่ามัธยฐาน (median) ของแต่ละค่าตั้ง สำหรับ n เล็ก โปรแกรมคูณซ้ำภายในหลายรอบ (`-r 200` ที่ n = 100 ลดลงตามลำดับจนเหลือ 1 รอบที่ n ≥ 3200) เพื่อให้ช่วงเวลาที่จับยาวพอเมื่อเทียบกับความละเอียดของนาฬิกา ข้อควรระวังคือ C ไม่ได้ถูกเติมค่าใน init หน้าของ C จึงถูกแตะครั้งแรกใน kernel เมื่อคูณเพียงรอบเดียว (n ≥ 3200) เวลา compute จึงรวม page fault ของ C ไว้ด้วย (20,000 ครั้งที่ n = 3200)

ด้านหน่วยความจำ โปรแกรมอ่าน VmHWM (RSS สูงสุด) และ VmPeak (ขนาด virtual สูงสุด) จาก `/proc/self/status` อ่าน AnonHugePages จาก `/proc/self/smaps_rollup` [13] และอ่านจำนวน page fault กับ context switch จาก `getrusage()` [14] ความต่างระหว่าง virtual กับ resident เป็นแก่นของข้อสังเกตหลายข้อ virtual memory คือช่วงที่อยู่ที่โปรเซสจองไว้ใน address space ซึ่งยังไม่จำเป็นต้องมีหน้าจริงใน RAM ส่วน resident memory (RSS) คือหน้าที่อยู่ใน RAM จริง kernel จัดหน้าจริงให้เมื่อโปรแกรมแตะหน้านั้นเป็นครั้งแรกผ่าน page fault (demand paging) [15]

ด้านความปลอดภัย โปรแกรมปฏิเสธการรันที่ต้องใช้หน่วยความจำเกิน 50% ของ RAM และจำกัดจำนวน thread ไม่เกิน 256 ตัว การปรับค่าตั้งของ OS ทุกอย่างทำเฉพาะกับโปรเซสที่ทดลอง (ตัวแปรสภาพแวดล้อม, taskset และ madvise) ไม่มีการแก้ค่าของระบบ ชุดการทดลองทั้งหมดสรุปในตารางที่ 3

ตารางที่ 3 ชุดการทดลองใน `bench.py`

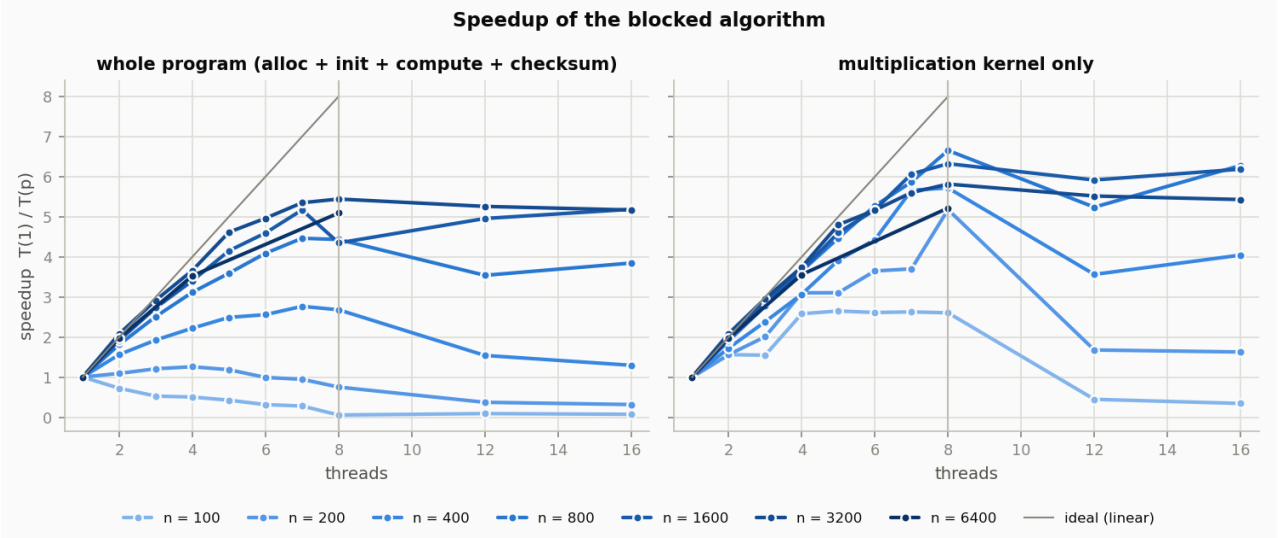
ชุด	สิ่งที่เปลี่ยน	ค่าที่ใช้
scaling	อัลกอริทึม × n × จำนวน thread	4 อัลกอริทึม, n = 100–3200 (blocked ถึง 6400), p = 1–8, 12, 16
parinit	init แบบขนาน (`-p`)	blocked, n = 100–3200, p = 1–8
block	ขนาด tile	bs = 8–512, n = 1600 และ 3200, p = 1 และ 8
pow2	n ใกล้กำลังของสอง	n = 1000–1048 และ 2000–2096, p = 1 และ 8
schedule	ชนิดและ chunk ของ schedule	static / dynamic / guided × chunk 0, 1, 4, 16; n = 400, 1600; p = 7, 8
compiler	flag ของ gcc	-O0, -O1, -O2, -O3, -O3 -march=native และเพิ่ม -ffast-math
osbind	ตัวแปรของ OpenMP runtime	OMP_PROC_BIND, OMP_PLACES, OMP_WAIT_POLICY; n = 200, 800, 3200; p = 4, 8, 16
affinity	CPU ที่อนุญาตด้วย taskset	1, 2, 4, 8 CPU × p = 1, 2, 4, 8, 16; n = 1600
thp	huge pages ผ่าน madvise	default / on / off; n = 800–6400

1.4 ผลการทดลองและการวิเคราะห์

1.4.1 (a) จาก 1 thread เป็นหลาย thread

ตารางที่ 4 speedup ของทั้งโปรแกรม T(1)/T(p) ของ blocked (init แบบ serial, median 5 รอบ)

n	p=2	p=3	p=4	p=5	p=6	p=7	p=8	p=12	p=16	T(1)
100	0.72	0.53	0.50	0.42	0.31	0.28	0.05	0.09	0.07	0.12 ms
200	1.09	1.21	1.26	1.18	0.99	0.95	0.75	0.37	0.31	0.58 ms
400	1.56	1.92	2.22	2.49	2.56	2.76	2.68	1.54	1.29	3.95 ms
800	1.81	2.51	3.12	3.59	4.09	4.46	4.43	3.54	3.84	31.1 ms
1600	1.92	2.73	3.40	4.15	4.59	5.17	4.35	4.95	5.18	0.250 s
3200	2.08	2.92	3.66	4.61	4.96	5.35	5.44	5.26	5.17	2.19 s
6400	1.98	–	3.53	–	–	–	5.09	–	–	17.0 s



รูปที่ 1 speedup ของ blocked ตามจำนวน thread ทั้งโปรแกรม (ซ้าย) และเฉพาะ kernel การคูณ (ขวา) เส้นแนวตั้งคือจำนวน CPU (8) ที่มา: results/plots/02_speedup.png

speedup สูงสุดของทั้งโปรแกรมคือ 5.44 เท่าที่ n = 3200 กับ 8 thread (ตารางที่ 4) และของ kernel อย่างเดียว คือ 6.65 เท่าที่ n = 800 แต่ไม่มีกรณีใดถึง 8 เท่า สาเหตุแบ่งได้สามกลุ่ม กลุ่มแรกคือส่วน serial (alloc, init และ checksum) ที่ไม่เร็วขึ้นตามจำนวน thread ตาม Amdahl's law ซึ่งวิเคราะห์ในหัวข้อ 1.5 กลุ่มที่สองเกิดขึ้นแม้ใน kernel ที่ขนานทั้งหมด ที่ n = 3200 blocked ทำได้ 30.6 GFLOP/s ด้วย thread เดียว แต่ 178.1 GFLOP/s ด้วย 8 thread หรือเฉลี่ยเพียง 22.3 GFLOP/s ต่อ thread (73%) สาเหตุที่น่าจะเป็นคือความถี่ขณะทุกคอร์ทำงานต่ำกว่าความถี่ boost ของคอร์เดียว [3] ทุกคอร์ใช้ L3 และ memory bandwidth ร่วมกัน และ thread ที่เสร็จก่อนต้องรอ thread ที่ช้าที่สุดที่ barrier ท้าย parallel region กลุ่มที่สามมาจากสภาพแวดล้อม WSL2 ซึ่งเห็นชัดที่ p = 8 พอดี

ความผิดปกติที่ p = 8 ข้อมูลรายรอบแสดงว่าเมื่อจำนวน thread เท่ากับจำนวน CPU พอดี การรันบางรอบช้าผิดปกติ เฟส spawn มีค่ามัธยฐาน 0.19 ms ที่ 4 thread และ 0.36 ms ที่ 7 thread แต่ที่ 8 thread มัธยฐานกระโดดเป็น 1.65 ms และ 10% ของการรันใช้เวลาเกิน 15.8 ms ขณะที่ 12 และ 16 thread อยู่ที่ 1.1 และ 1.5 ms อย่างสม่ำเสมอ เวลารวมของ n = 100 ที่ p = 8 จาก 5 รอบคือ 0.47, 0.47, 2.23, 15.32 และ 16.31 ms ขนาดของการกระโดดราว 15–16 ms ใกล้เคียงกับคาบ timer มาตรฐานของ Windows (15.6 ms) จึงน่าจะเป็นจังหวะที่ vCPU ตัวหนึ่งถูก host พักไปหนึ่งคาบ คำอธิบายที่สอดคล้องกับทุกข้อมูลคือ เมื่อ p = 8 libgomp ให้ thread ที่รอ spin ได้ 300,000 รอบ [6] ทุก vCPU จึงไม่ว่างเลย งานใดๆ ที่ต้องการ CPU (ทั้งใน VM และบน host) ต้องแย่งเวลาจาก thread ของโปรแกรม และ thread ที่ถูกพักทำให้ทุก thread ต้องรอที่ barrier ส่วนที่ $p \le 7$ ยังมี CPU ว่าง และที่ $p \ge 12$ libgomp ลดการ spin เหลือ 100 รอบเพราะ thread มากกว่า CPU thread ที่รอจึงคืน CPU เร็ว จำนวน involuntary context switch ที่ n = 1600 สนับสนุนคำอธิบายนี้ คือ 4 ครั้งที่ p = 7 แต่ 14 ครั้งที่ p = 8

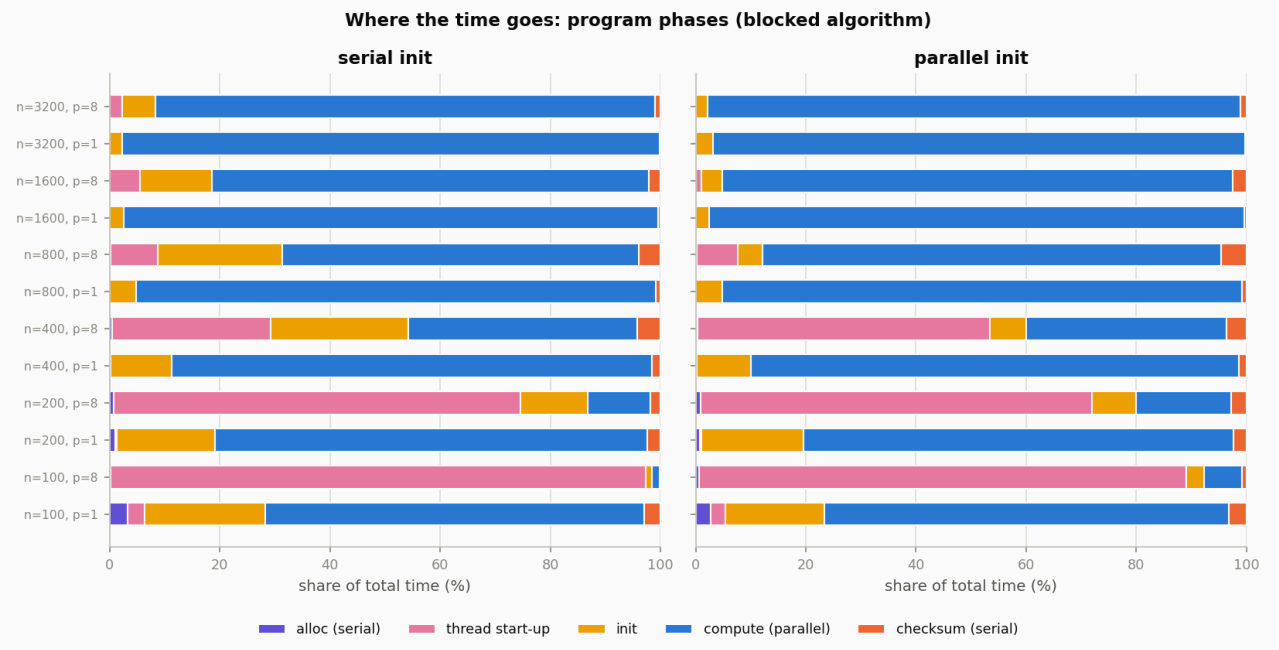
ผลนี้ทำให้ต้องอ่านตัวเลข "p = 7 ดีกว่า p = 8" ที่ n = 1600 (5.17 เทียบกับ 4.35 เท่า) อย่างระมัดระวัง เวลา kernel ล้วนที่ p = 8 (มัธยฐาน 38.4 ms, speedup 6.32 เท่า) เร็วกว่า p = 7 (40.0 ms, 6.07 เท่า) ด้วยซ้ำ แต่เวลารวมของ 5 รอบที่ p = 8 กระจายตั้งแต่ 41.7 ถึง 70.2 ms ค่ามัธยฐานของเวลารวมจึงสะท้อนการรบกวนมากกว่าความสามารถในการขยายตัวของโปรแกรม

เมื่อเพิ่มเกิน 8 thread (12 หรือ 16) ไม่มีทางเร็วขึ้นเพราะมี CPU ให้ใช้เพียง 8 ตัว ที่ n = 3200 speedup ลดจาก 5.44 เหลือ 5.26 และ 5.17 เท่า และจำนวน involuntary context switch เพิ่มจาก 13 ครั้ง (p = 8) เป็น 248 และ 554 ครั้ง ซึ่งเป็นหลักฐานโดยตรงของการแบ่งเวลา (time slicing) ระหว่าง thread ที่มากกว่า CPU ส่วนที่ n

≤ 400 การใช้ 12–16 thread แย่ลงชัดเจน เพราะเวลา spawn (1.1–1.5 ms) ใหญ่เมื่อเทียบกับงาน

superlinear speedup ที่ n = 3200, p = 2 ได้ 2.08 เท่าทั้งในเวลารวมและใน kernel ข้อมูลรายรอบชี้สาเหตุสองส่วน ส่วนแรกคือการรัน p = 1 เสียเปรียบโดยไม่เกี่ยวกับการขนาน: เฟส init ซึ่งเป็น serial ในทุกกรณีใช้เวลา 33–65 ms ที่ p = 1 แต่เพียง 22–29 ms ที่ p ≥ 2 และที่ n = 6400 ใช้ 244–265 ms เทียบกับ 89–99 ms ปรากฏการณ์เดียวกันเกิดกับ ikj และ transpose ที่ n = 3200 ด้วย เนื่องจาก bench.py รันค่าตั้งเรียงตามจำนวน thread การรัน p = 1 จึงเป็นโปรเซสแรกที่แตะหน่วยความจำขนาดใหญ่นี้หลังจากรันขนาดเล็ก ซึ่งใน WSL2 อาจต้องรอให้ host จัดหน้าจริงให้ VM ก่อน การรันถัดไปใช้หน้าที่ VM ได้มาแล้ว เมื่อ kernel ที่ n = 3200 รวม page fault ของ C ไว้ด้วย (หัวข้อ 1.3) T(1) จึงยาวเกินจริงเล็กน้อย ส่วนที่สองคือเมื่อใช้ 2 คอร์ ความจุของ L1 และ L2 ส่วนตัวรวมกันเพิ่มเป็นสองเท่า ข้อมูลเฉพาะของแต่ละ thread จึงหลุดจาก cache น้อยลง เนื่องจากส่วนเกินเพียง 4% และมีสาเหตุแรกปนอยู่ จึงยังสรุปไม่ได้ว่าเป็น superlinear speedup ที่แท้จริง ที่ n = 6400 speedup ของ p = 2 เท่ากับ 1.98 ซึ่งเป็นปกติ

1.4.2 (b) จากขนาดเล็กไปขนาดใหญ่

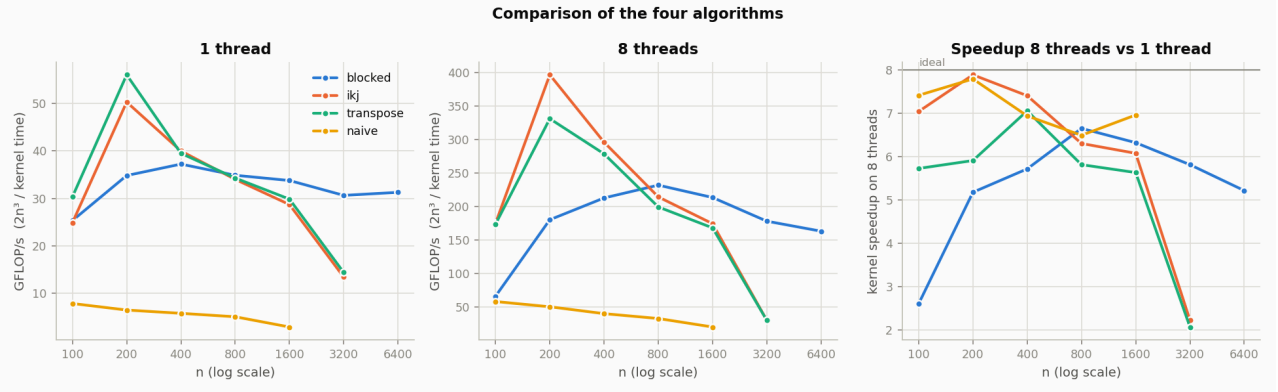


รูปที่ 2 สัดส่วนเวลาของแต่ละเฟส (alloc, สร้าง thread, init, compute, checksum) ที่ p = 1 และ p = 8 สำหรับ init แบบ serial (ซ้าย) และแบบขนาน (ขวา) ที่มา: results/plots/07_time_breakdown.png

รูปที่ 2 อธิบายว่าทำไม n เล็กแทบไม่ได้ประโยชน์จาก thread ที่ n = 100 kernel ใช้เวลาเพียง 0.079 ms ด้วย thread เดียว และ 0.030 ms ด้วย 8 thread แต่การสร้างทีม 8 thread ใช้มัธยฐาน 2.1 ms ทั้งโปรแกรมจึงช้ากว่าแบบ thread เดียวถึง 20 เท่า (speedup 0.05) อีกทั้ง blocked มีเพียง 4 tile จึงใช้ได้แค่ 4 thread ที่ n = 200 kernel ลดจาก 0.46 เป็น 0.09 ms แต่ spawn ใช้ 0.58 ms speedup ที่ดีที่สุดจึงเพียง 1.26 เท่าที่ p = 4 และต่ำกว่า 1 ตั้งแต่ p = 6 ขึ้นไป ที่ n = 400 ทำได้สูงสุด 2.76 เท่าที่ p = 7 ส่วน n ≥ 800 เวลา compute ครอบงำเฟสอื่น ความแปรปรวนของเวลาก็สูงที่สุดที่ n เล็กกับ 8 thread เช่น coefficient of variation ของเวลารวมของ naive ที่ n = 200, p = 8 สูงถึง 704% และของ blocked 574% เพราะการกระโดด 15 ms เพียงครั้งเดียวใหญ่กว่างานทั้งหมดหลายสิบเท่า

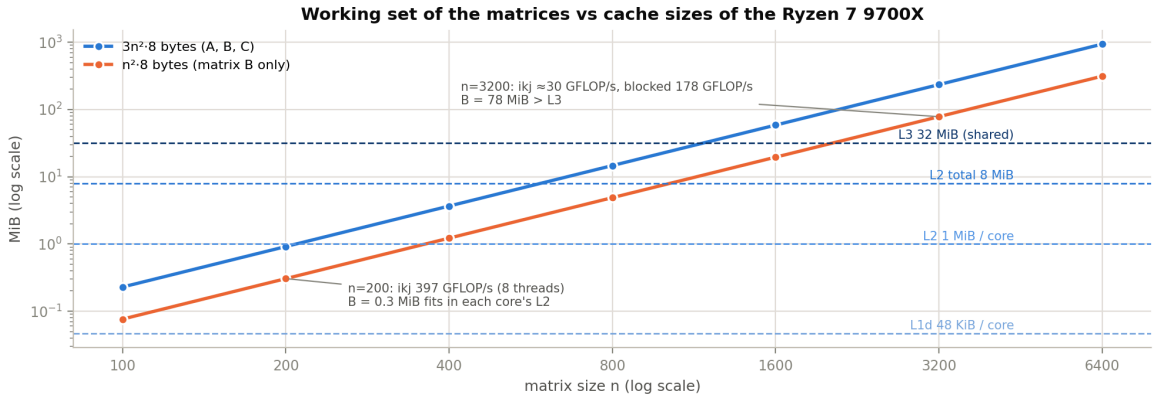
ตารางที่ 5 ความเร็วของ kernel (GFLOP/s) ที่ 1 thread / 8 thread

อัลกอริทึม	n=100	n=200	n=400	n=800	n=1600	n=3200	n=6400
blocked	25.3 / 65.9	34.8 / 180.2	37.2 / 212.8	34.9 / 232.0	33.8 / 213.5	30.6 / 178.1	31.3 / 163.1
ikj	24.9 / 175.1	50.3 / 397.0	40.0 / 296.2	34.0 / 214.3	28.7 / 174.3	13.4 / 29.7	–
transpose	30.3 / 173.6	56.1 / 331.3	39.5 / 278.7	34.3 / 199.3	29.8 / 168.0	14.4 / 29.7	–
naive	7.8 / 57.9	6.4 / 50.1	5.7 / 39.8	5.0 / 32.5	2.8 / 19.8	–	–



รูปที่ 3 เปรียบเทียบสี่อัลกอริทึม: GFLOP/s ที่ 1 thread และ 8 thread และ speedup ของ kernel จาก 1 เป็น 8 thread ที่มา: results/plots/04_algorithms.png

ตารางที่ 5 และรูปที่ 3 แสดงว่าอัลกอริทึมที่ดีที่สุดเปลี่ยนไปตาม n เมื่อข้อมูลจุใน cache (n ≤ 400) ikj และ transpose เร็วกว่า blocked เพราะ loop เรียบง่ายกว่าและไม่มีค่าใช้จ่ายของการแบ่ง tile โดย ikj ทำได้ถึง 397 GFLOP/s ที่ n = 200 กับ 8 thread แต่ที่ n = 3200 ikj และ transpose ตกลงเหลือราว 30 GFLOP/s และ speedup จาก 1 เป็น 8 thread เพียงประมาณ 2.2 เท่า ขณะที่ blocked ยังได้ 178 GFLOP/s รูปที่ 4 เทียบขนาดข้อมูลกับขนาด cache ที่ n = 200 เมทริกซ์ B มีขนาดเพียง 0.3 MiB จุใน L2 ของแต่ละคอร์ (1 MiB) ทุก thread จึงอ่าน B ซ้ำจาก L2 ได้ตลอด แต่ที่ n = 3200 B มีขนาด 78 MiB ใหญ่กว่า L3 ทั้งก้อน (32 MiB) ทุกครั้งที่ ikj หรือ transpose เริ่มแถว i ใหม่จึงต้องอ่าน B (หรือ B^T) ทั้งหมดจาก DRAM ซึ่งมี bandwidth จำกัดและใช้ร่วมกันทุกคอร์ การเพิ่ม thread จึงช่วยได้น้อย



รูปที่ 4 ขนาดของเมทริกซ์ทั้งสาม ($3n^2 \cdot 8$ ไบต์) และของ B อย่างเดียว ($n^2 \cdot 8$ ไบต์) เทียบกับความจุ cache ของ Ryzen 7 9700X (สเกล log ทั้งสองแกน)

Roofline model [16] อธิบายเรื่องนี้ด้วย arithmetic intensity (AI) คือจำนวน FLOP ต่อไบต์ที่ต้องโหลดจากหน่วยความจำระดับที่ช้า ถ้า AI ต่ำ ความเร็วจะถูกจำกัดด้วย bandwidth ไม่ใช่ความสามารถในการคำนวณ ตารางที่ 6 ประมาณ AI แบบหยาบจาก loop ในสุด เมื่อข้อมูลใหญ่กว่า cache

ตารางที่ 6 arithmetic intensity โดยประมาณเมื่อข้อมูลใหญ่กว่า cache

อัลกอริทึม	ข้อมูลที่ต้องโหลดต่อ FMA หนึ่งครั้ง (2 FLOP)	AI (FLOP/ไบต์)	ตัวจำกัดความเร็ว
naive	B หนึ่ง cache line (64 ไบต์) เมื่อคอลัมน์ไม่จุใน cache	≈ 2/64 ≈ 0.03	latency และ bandwidth ของหน่วยความจำ
ikj, transpose	B หรือ B^T หนึ่ง double (8 ไบต์)	≈ 2/8 = 0.25	memory bandwidth
blocked (bs = 64)	B หนึ่ง double ต่อการใช้ซ้ำ bs ครั้ง	≈ 2·bs/8 = 16	ความสามารถในการคำนวณ (SIMD และ FMA)

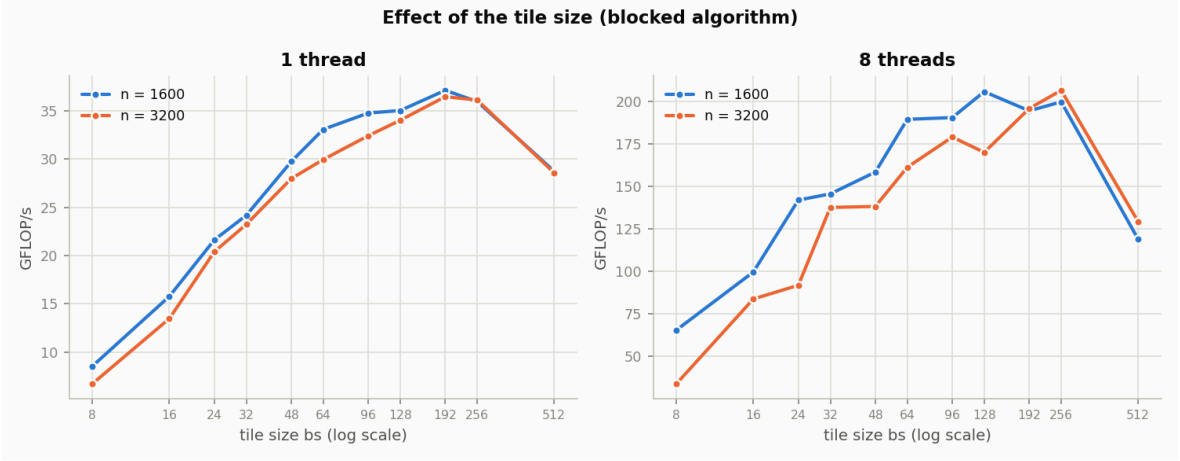
ikj มี AI ต่ำกว่า blocked ราว 64 เท่า จึงอยู่ฝั่ง memory-bound ของ roofline เมื่อข้อมูลไม่จุใน cache ส่วน blocked ใช้ข้อมูลแต่ละไบต์ซ้ำหลายสิบครั้งจึงอยู่ฝั่ง compute-bound ความเร็วต่อ thread ของ blocked จึงคงที่ราว 31–37 GFLOP/s ในทุกขนาด naive ช้าที่สุดในทุกขนาด (เหลือ 2.8 GFLOP/s ที่ n = 1600) แต่ที่น่าสังเกตคือ naive ได้ speedup สูงถึงราว 7 เท่าจาก 8 thread (2.8 → 19.8 GFLOP/s) เพราะคอขวดของ naive คือเวลาแฝงของการรอ cache miss ทีละครั้ง เมื่อมีหลายคอร์ จำนวน cache miss ที่รอพร้อมกันได้ก็เพิ่มตาม ต่างจาก ikj ที่ n = 3200 ซึ่งใช้ bandwidth ของ DRAM จนเต็มแล้ว

1.4.3 (c) ค่าตั้งของโปรแกรม

ขนาด tile ของ blocked

ตารางที่ 7 ความเร็วของ blocked (GFLOP/s) ตามขนาด tile และขนาดของ tile เทียบกับ cache

n, p	bs=8	16	32	64	96	128	192	256	512
1600, 1	8.5	15.8	24.2	33.1	34.8	35.0	**37.1**	36.0	28.7
3200, 1	6.7	13.5	23.2	29.9	32.4	34.0	**36.5**	36.1	28.6
1600, 8	65.4	99.7	145.7	189.7	190.7	**206.0**	194.7	200.0	119.0
3200, 8	33.8	83.7	137.7	161.4	179.2	170.2	196.1	**206.8**	129.4
tile ของ B ($bs^2 \cdot 8$)	0.5 KiB	2 KiB	8 KiB	32 KiB	72 KiB	128 KiB	288 KiB	512 KiB	2 MiB
อยู่ใน	L1	L1	L1	L1	L2	L2	L2	L2	L3



รูปที่ 5 ผลของขนาด tile ต่อความเร็วของ blocked ที่ 1 thread และ 8 thread ที่มา: results/plots/09_block_size.png

bs ที่ดีที่สุดอยู่ในช่วง 128–256 (ตารางที่ 7) bs = 8 ช้ากว่าค่าที่ดีที่สุด 3.1–6.1 เท่า และ bs = 512 ตกลงอีกครั้ง เมื่อ bs = 8 loop ในสุด (ตาม j) ยาวเพียง 8 รอบ เท่ากับ vector AVX-512 แค่ตัวเดียว ค่าใช้จ่ายของการเริ่มและจบ loop จึงมากเมื่อเทียบกับงาน และแถวย่อยของ C ถูกอ่านและเขียนใหม่ทุกช่วงของ k (n/bs = 200–400 ครั้ง) เมื่อ bs โตขึ้นค่าใช้จ่ายเหล่านี้ลดลง และตราบใดที่ tile ของ B ซึ่งถูกใช้ซ้ำ bs ครั้งยังอยู่ใน L2 ความเร็วจะยังสูง เพราะ L2 ของ Zen 5 ส่งข้อมูลให้ L1 ได้ 64 ไบต์ต่อรอบสัญญาณนาฬิกา [4] ขณะที่ค่า a และแถวย่อยของ C ถูกใช้ซ้ำในรีจิสเตอร์และ L1 [10] bs = 256 เป็นขนาดใหญ่สุดที่ tile ของ B (512 KiB) ยังจุใน L2 ส่วน bs = 512 ทำให้ tile ของ B ใหญ่ 2 MiB เกิน L2 ต้องดึงจาก L3 ความเร็วจึงลดลง 22% ที่ 1 thread แต่ลดลงถึง 37–42% ที่ 8 thread เพราะทั้ง 8 คอร์ดึงข้อมูลจาก L3 ก้อนเดียวกันพร้อมกัน ค่าเริ่มต้น bs = 64 ที่ใช้ในการทดลองอื่นช้ากว่าค่าที่ดีที่สุด 8–22% แต่มีข้อดีคือให้จำนวน tile มากพอสำหรับ n เล็ก

n ที่เป็นกำลังของสอง

ตารางที่ 8 ความเร็ว (GFLOP/s, 1 thread / 8 thread) ของ n ใกล้กำลังของสอง

อัลกอริทึม	n=1000	n=1016	n=1024	n=1048	n=2000	n=2048	n=2064
naive	4.5 / 27.3	4.5 / 31.5	**0.8 / 4.3**	4.4 / 30.6	–	–	–
ikj	32.7 / 194.4	32.2 / 192.6	31.5 / 193.5	32.9 / 147.2	17.8 / 87.4	**18.0 / 68.9**	16.9 / 77.8
blocked	34.3 / 233.4	35.0 / 241.1	**30.7 / 206.8**	34.1 / 195.8	33.6 / 176.8	**30.8 / 167.1**	33.9 / 191.3

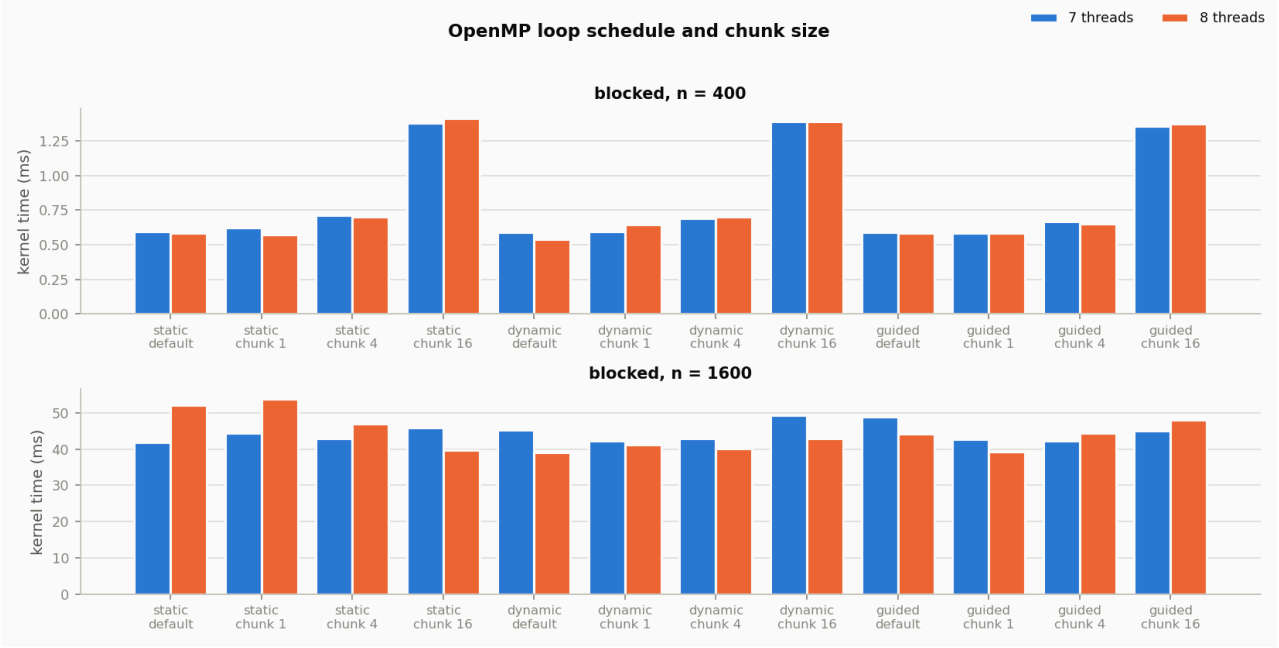
ผลที่รุนแรงที่สุดเกิดกับ naive ที่ n = 1024 ซึ่งได้เพียง 0.8 GFLOP/s เทียบกับ 4.5 GFLOP/s ที่ n = 1000 (ช้ากว่า 5.6 เท่าที่ 1 thread และ 6.3 เท่าที่ 8 thread) ทั้งที่งานต่างกันเพียง 7% สาเหตุคือ cache แบบ set-associative เลือก set ของแต่ละ line จากบิตกลางของที่อยู่ [8], [9] L1d ขนาด 48 KiB แบบ 12-way มี 48 KiB / (64 × 12) = 64 set ซึ่งใช้บิตที่ 6–11 ของที่อยู่ ส่วน L2 ขนาด 1 MiB แบบ 16-way มี 1,024 set ซึ่งใช้บิตที่ 6–15 (วนครบรอบทุก 64 KiB) ตารางที่ 9 คำนวณจำนวน set ที่หนึ่งคอลัมน์ของ B ใช้ได้

ตารางที่ 9 การกระจายของหนึ่งคอลัมน์ของ B ลงใน set ของ cache

	n = 1000	n = 1024
ระยะห่างระหว่างสมาชิกในคอลัมน์	8,000 ไบต์	8,192 ไบต์ = 2^{13}
set ของ L1 ที่คอลัมน์ใช้	กระจายทั้ง 64 set	1 set (12 line)
set ของ L2 ที่คอลัมน์ใช้	กระจายทั้ง 1,024 set	8 set (8 × 16 = 128 line)
จำนวน line ที่คอลัมน์ต้องการ	1,000	1,024
ผล	คอลัมน์จุใน L2 ได้ทั้งหมด	จุไม่ได้แม้ใน L2

naive ใช้ line ของคอลัมน์ j ซ้ำกับคอลัมน์ j+1 ถึง j+7 ที่อยู่ใน line เดียวกัน ถ้า line ยังอยู่ใน cache การอ่าน 7 ใน 8 ครั้งจะ hit ที่ n = 1000 คอลัมน์ 1,000 line กระจายทั่วทุก set จึงอยู่ใน L2 ได้ทั้งหมด แต่ที่ n = 1024 ทั้ง 1,024 line ต้องแย่งกันอยู่ใน L2 เพียง 8 set (128 ช่อง) จึงไล่กันเองตลอดเวลา (conflict miss) ทุกการอ่านต้องไปถึง L3 สำหรับ blocked ผลอ่อนกว่ามาก คือช้าลงราว 5–14% ที่ n = 1024 และ 2048 ทั้ง 1 และ 8 thread เพราะแถวทั้ง bs แถวของ tile ห่างกัน n·8 ไบต์ จึงแย่ง set ชุดเดียวกัน แต่ tile แต่ละแถวยาว 8 line ติดกันซึ่งกระจายไปหลาย set ได้ ส่วน ikj ได้รับผลเฉพาะที่ 8 thread และ n = 2048 (ลดลง 12–21% เทียบกับขนาดข้างเคียง) วิธีแก้ที่นิยมคือเติมความกว้างของแถว (padding) ไม่ให้เป็นกำลังของสอง [10] ข้อสังเกตเพิ่มเติมคือ ikj และ blocked ที่ n = 1032 และ 1048 กับ 8 thread ก็ช้าลงด้วย (เช่น blocked 135.7 GFLOP/s ที่ n = 1032) ทั้งที่ค่า 1 thread ปกติและ n ไม่ใช่กำลังของสอง ผลนี้อธิบายด้วย cache ไม่ได้ จึงน่าจะเป็นการรบกวนจาก host ในรอบที่วัด (ชุดนี้วัดเพียง 3 รอบ)

OpenMP schedule



รูปที่ 6 เวลา kernel ของ blocked ตามชนิดของ schedule และขนาด chunk ที่ n = 400 (บน) และ n = 1600 (ล่าง) ที่มา: ส่วนบนของ results/plots/11_schedule.png

สำหรับ blocked ที่ n = 400 ชนิดของ schedule แทบไม่มีผล แต่ขนาด chunk มีผลมาก chunk = 16 ทำให้ kernel ช้าลงจาก 0.60 เป็น 1.38 ms (2.3 เท่า) ทุกชนิดของ schedule เหตุผลอยู่ที่จำนวนงาน n = 400 กับ bs = 64 มีเพียง 49 tile เมื่อตัดเป็น chunk ละ 16 จะได้แค่ 4 chunk (16, 16, 16 และ 1 tile) มีเพียง 4 thread ที่ได้

งาน และ thread ที่ได้ 16 tile เป็นผู้กำหนดเวลา ขณะที่ static ปกติแบ่งให้ thread ละ 7 tile (p = 7) อัตราส่วน 16/7 ≈ 2.29 ตรงกับที่วัดได้ การคำนวณเดียวกันทำนาย chunk = 4 ได้ด้วย คือ 13 chunk ทำให้บาง thread ได้ 2 chunk หรือ 8 tile ช้าลง 8/7 ≈ 1.14 เท่า ซึ่งใกล้กับ 1.18 เท่าที่วัดได้ (0.71 เทียบกับ 0.60 ms) ข้อสรุปคือขนาด chunk ต้องเล็กเมื่อเทียบกับจำนวนงานต่อ thread

ที่ n = 1600 (625 tile) ความต่างอยู่ในระดับ noise เป็นส่วนใหญ่ แต่มีรูปแบบที่สอดคล้องกับหัวข้อ 1.4.1: ที่ p = 8 static แบบปกติใช้ 52.0 ms ขณะที่ dynamic และ guided ส่วนใหญ่ใช้ 39–44 ms ส่วนที่ p = 7 static ปกติ (41.9 ms) ดีพอๆ กับแบบอื่น static แบ่งงานตายตัวล่วงหน้า ถ้า vCPU ตัวหนึ่งถูกพัก งานของ thread นั้นต้องรอจนกว่าจะได้ CPU คืน ขณะที่ dynamic ให้ thread ที่ว่างมารับงานที่เหลือแทนได้ สำหรับ ikj ที่ n = 1600 dynamic แบบ chunk 1 กลับช้ากว่า static 20–45% (62–75 เทียบกับ 51–52 ms) เพราะ ikj ที่ขนาดนี้ถูกจำกัดด้วย bandwidth และการแจกแถวทีละแถวทำให้ thread ไม่ได้เดินไปตามแถวของ B พร้อมๆ กัน จึงใช้ข้อมูลที่ thread อื่นเพิ่งดึงเข้า L3 ร่วมกันได้น้อยลง (ข้อนี้เป็นสมมติฐานที่ยังไม่ได้วัดยืนยัน)

flag ของ compiler

ตารางที่ 10 ความเร็ว (GFLOP/s) ตาม flag ของ gcc 13.3

อัลกอริทึม, n, p	-O0	-O1	-O2	-O3	-O3 -march=native	+ -ffast-math
blocked, 1600, 1	2.37	8.70	5.68	17.89	33.35	34.26
ikj, 1600, 1	2.14	8.49	9.99	17.04	26.73	24.73
naive, 800, 1	0.91	4.87	4.85	4.91	4.72	4.78
blocked, 1600, 8	14.33	55.13	36.13	105.66	197.49	214.15
ikj, 1600, 8	13.03	52.84	63.28	112.82	145.11	173.52
naive, 800, 8	6.50	28.34	33.32	33.39	32.53	27.09

blocked ด้วย thread เดียวเร็วขึ้นจาก 2.37 GFLOP/s ที่ -O0 เป็น 17.89 ที่ -O3 และ 33.35 ที่ -O3 -march=native หรือรวม 14 เท่า (ตารางที่ 10) ที่ -O0 ทุกตัวแปรถูกอ่านและเขียนผ่านหน่วยความจำและไม่มีการแปลงเป็น vector -O1 เก็บตัวแปรในรีจิสเตอร์จึงเร็วขึ้นราว 4 เท่า -O3 ทำ vectorization ได้แต่ต้องใช้ชุดคำสั่งพื้นฐานของ x86-64 คือ SSE2 ขนาด 128 บิต (double 2 ตัวต่อคำสั่ง) และไม่มี FMA ขณะที่ -march=native เปิดใช้ AVX-512 (double 8 ตัวต่อคำสั่ง) และ FMA จึงเร็วขึ้นอีกเกือบสองเท่า ส่วน -ffast-math เพิ่มอีกเพียง 3–8% สำหรับ blocked เพราะ loop ของ blocked ไม่มี reduction ที่ต้องสลับลำดับอยู่แล้ว ข้อสังเกตที่ไม่คาดคิดคือ blocked ที่ -O2 (5.68) ช้ากว่า -O1 (8.70) ซึ่งน่าจะมาจากการแปลง loop ที่ต่างกันระหว่างสองระดับ สาเหตุที่แน่ชัดต้องตรวจจาก assembly

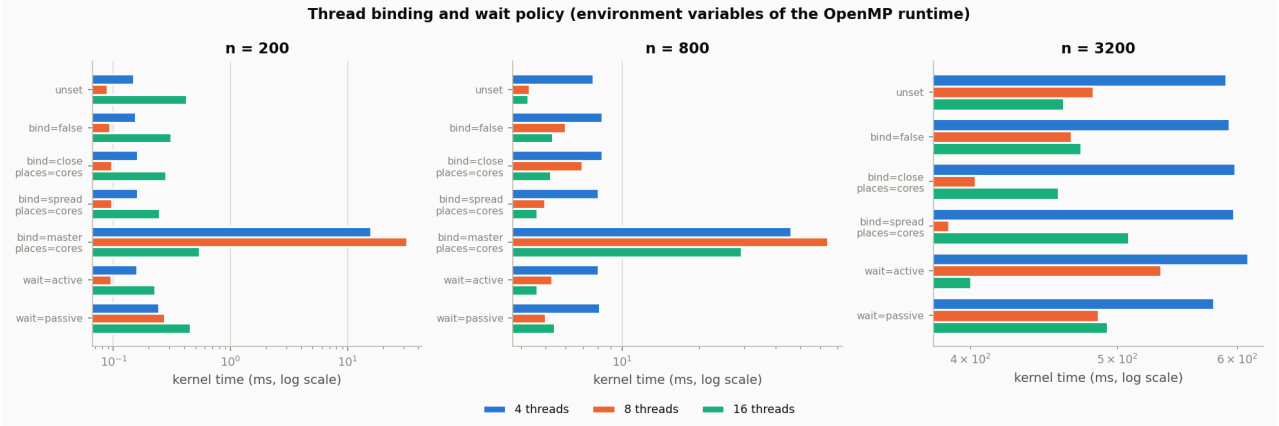
naive ต่างออกไป: ได้ประโยชน์เฉพาะจาก -O0 ไป -O1 (0.91 → 4.87) แล้วคงที่ราว 4.7–4.9 GFLOP/s ไม่ว่าจะเปิด vectorization หรือ AVX-512 และแม้แต่ -ffast-math ที่อนุญาตให้ vectorize reduction ได้ก็ไม่ช่วย เพราะคอขวดของ naive คือการรอข้อมูลจากหน่วยความจำ (stride n) ไม่ใช่จำนวนคำสั่งคำนวณ ผลนี้สอดคล้องกับ roofline คือการเพิ่มความสามารถในการคำนวณช่วยเฉพาะโค้ดที่เป็น compute-bound

1.4.4 (d) ค่าตั้งของระบบปฏิบัติการและ OpenMP runtime

การทดลองนี้เปลี่ยนตัวแปรสภาพแวดล้อมของ OpenMP runtime [6] ใช้ taskset จำกัด CPU ที่โปรเซสใช้ได้ [14] และขอ huge pages ผ่าน madvise() [7]

ตารางที่ 11 เวลา kernel ของ blocked (ms) ตามค่าตั้งของ OpenMP runtime

ค่าตั้ง	n	p=4	p=8	p=16	involuntary ctx switch (p=16)
unset	200	0.15	0.09	0.42	3
bind=close, places=cores	200	0.16	0.10	0.28	7
bind=spread, places=cores	200	0.16	0.10	0.25	4
bind=master, places=cores	200	**15.89**	**31.98**	0.54	69
wait=active	200	0.16	0.10	0.23	1
wait=passive	200	0.24	**0.27**	0.45	1
unset	800	7.69	4.32	4.26	5
bind=master, places=cores	800	46.04	64.00	29.42	8
unset	3200	590.2	482.3	461.2	507
bind=false	3200	593.1	466.5	473.6	599
bind=close, places=cores	3200	598.3	403.1	457.5	710
bind=spread, places=cores	3200	597.0	387.2	509.3	687



รูปที่ 7 เวลา kernel ของ blocked ตามค่า OMP_PROC_BIND, OMP_PLACES และ OMP_WAIT_POLICY (สเกล log) ที่มา: results/plots/13_os_binding.png

bind=master สั่งให้ทุก thread อยู่ใน place เดียวกับ master thread เมื่อ OMP_PLACES=cores ทั้งทีมจึงถูกผูกไว้กับ CPU ตัวเดียว ที่ n = 200 kernel ช้าลง 106 เท่าที่ p = 4 และ 355 เท่าที่ p = 8 (31.98 เทียบกับ 0.09 ms) ซึ่งมากกว่าการเสียความขนานไปเฉยๆ หลายสิบเท่า สาเหตุคือเมื่อไม่ได้ตั้ง OMP_WAIT_POLICY thread ที่รองานจะ spin คือวนตรวจสถานะโดยไม่คืน CPU ได้ถึง 300,000 รอบก่อนจะหลับ [6] thread ที่ spin จึงแย่งเวลาของ CPU ตัวเดียวกับ thread ที่มีงานจริง ทุก parallel region (ซึ่งมี 50 รอบที่ n = 200) ผลที่ p = 16 ยืนยันกลไกนี้ได้ชัดเจน: เวลากลับลดลงเหลือ 0.54 ms ใกล้กับเวลาของ thread เดียว (0.46 ms) เพราะเมื่อ thread มากกว่าจำนวน CPU ใน affinity mask (8) libgomp ลดการ spin เหลือ 100 รอบตามที่เอกสารระบุ thread ที่รอจึงหลับเกือบทันที ที่ n = 800 ภาพเดียวกันชัดยิ่งขึ้น: p = 16 ใช้ 29.42 ms ซึ่งเท่ากับเวลา kernel ของ thread เดียว (29.37 ms) แทบพอดี คืองานทั้งหมดทำบน CPU เดียวโดยไม่มีการ spin เปล่าเลย ส่วน p = 8 ใช้ 64.00 ms หรือนานกว่าการทำแบบลำดับ 2.2 เท่า ข้อนี้แสดงด้วยว่า runtime ตัดสินจากจำนวน CPU ใน affinity mask ของโปรเซส โดยไม่ได้คำนึงว่า place ที่ผูกไว้มี CPU เพียงตัวเดียว

bind=close และ spread ที่ n เล็กแทบไม่ต่างจาก unset ที่ n = 3200 กับ p = 8 close และ spread ใช้ 403 และ 387 ms เทียบกับ unset 482 ms ซึ่งดูเหมือนเร็วขึ้น 16–20% แต่ค่าตั้งแบบ unset เดียวกันในชุด scaling ใช้เพียง 368 ms แปลว่า unset ในชุดนี้ช้าเพราะการรบกวนในรอบที่วัด ไม่ใช่เพราะการไม่ผูก thread ข้อสรุปที่ข้อมูลรองรับได้จริงจึงมีเพียงว่า การผูก thread ไว้กับ CPU คนละตัวไม่ทำให้ช้าลงที่ p = 8 และอาจช่วยได้บ้างเพราะ thread ไม่ถูกย้ายข้าม CPU ข้อมูลใน L1 และ L2 จึงยังอยู่ ส่วนที่ p = 16 spread ช้ากว่า unset 10% (509 เทียบกับ 461 ms) ซึ่งสอดคล้องกับเหตุผลว่าเมื่อ thread มากกว่า CPU การผูกไว้ทำให้ scheduler ย้าย thread ไป CPU ที่ว่างกว่าไม่ได้ แต่ก็อยู่ในระดับ noise เช่นกัน บนชิปนี้ซึ่งมี L3 ก้อนเดียวและ NUMA node เดียว close กับ spread ไม่มีเหตุผลเชิงโครงสร้างที่จะต่างกันมาก และใน WSL2 การผูก thread ทำได้เพียงกับ vCPU ซึ่ง host ยังย้ายไปมาระหว่างคอร์จริงได้เอง

wait policy OMP_WAIT_POLICY=passive ทำให้ thread ที่รอหลับทันทีแทนการ spin ซึ่งประหยัด CPU แต่ทุกครั้งที่เริ่ม parallel region ใหม่ thread อื่นต้องถูกปลุกผ่าน system call ของ kernel ที่ n = 200 แต่ละ region ใช้เวลาไม่ถึง 0.1 ms ที่ 8 thread ค่าใช้จ่ายในการปลุกจึงทำให้ช้าลง 3 เท่า (0.27 เทียบกับ 0.09 ms) และ 1.6 เท่าที่ p = 4 ส่วนที่ p = 16 แทบไม่ต่าง (0.45 เทียบกับ 0.42 ms) เพราะ libgomp ลดการ spin ให้อยู่แล้วเมื่อ thread เกินจำนวน CPU และที่ n = 800 หรือ 3200 ซึ่งแต่ละ region ยาว ความต่างอยู่ในระดับ noise

ตารางที่ 12 เวลา kernel (s) ของ blocked n = 1600 เมื่อจำกัด CPU ด้วย taskset

CPU ที่อนุญาต	p=1	p=2	p=4	p=8	p=16
1	0.2408	0.2408	0.2446	0.2372	0.2299
2	0.2301	0.1217	0.1274	0.1254	0.1275
4	0.2508	0.1319	0.0664	0.0695	0.0698
8	0.2509	0.1324	0.0716	0.0441	0.0425

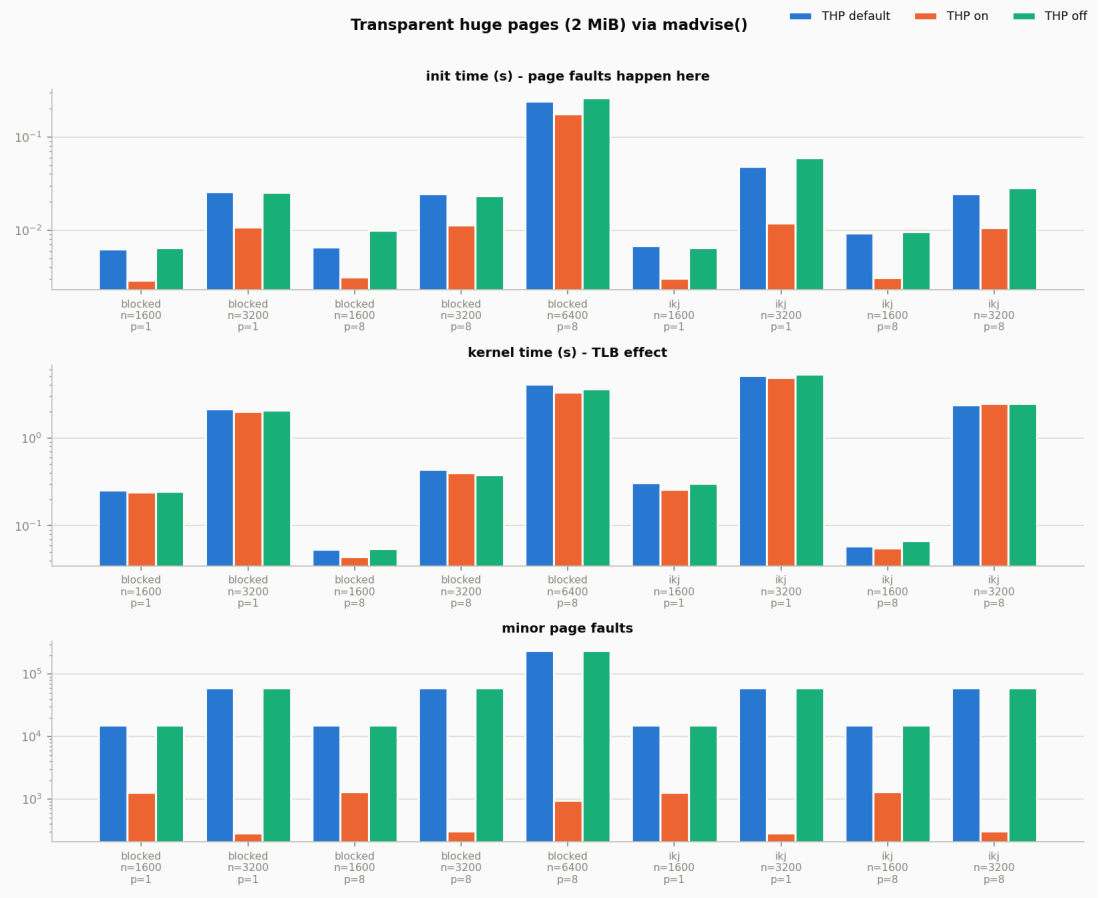
CPU affinity (taskset) ตารางที่ 12 แสดงว่าเวลาถูกกำหนดโดย min(จำนวน thread, จำนวน CPU) เกือบสมบูรณ์ เมื่ออนุญาต CPU เดียว ไม่ว่าจะสั่งกี่ thread เวลาอยู่ที่ราว 0.23–0.24 วินาทีเท่ากับ thread เดียว และ 8 thread บน 4 CPU (0.0695 s) ใกล้กับ 4 thread บน 4 CPU (0.0664 s) thread ที่เกินจำนวน CPU ไม่ได้ทำให้เร็วขึ้น เพราะ scheduler ของ OS แบ่งเวลาให้ thread สลับกันใช้ CPU งานรวมจึงเท่าเดิม ที่น่าสังเกตคือ oversubscription แทบไม่ทำให้ช้าลงเลย ต่างจากกรณี bind=master ที่ช้าลงหลายสิบเท่า ความต่างคือ taskset เปลี่ยน affinity mask ของโปรเซส libgomp จึงรู้ว่ามี CPU น้อยกว่า thread และลดการ spin ลงเหลือ 100 รอบ [6] ส่วน bind=master ไม่ได้เปลี่ยน mask

ตารางที่ 13 ผลของ transparent huge pages (median; default ≡ off เพราะระบบตั้งไว้เป็น madvise)

กรณี	THP	init (ms)	kernel (s)	GFLOP/s	AnonHugePages (MiB)	minor faults
blocked n=1600, p=1	default	6.3	0.2534	32.3	0	15,078
	on	2.9	0.2425	33.8	54	1,281
	off	6.4	0.2444	33.5	0	15,078
blocked n=3200, p=8	default	24.7	0.4337	151.1	0	60,095
	on	11.4	0.4003	163.7	234	308
	off	23.3	0.3816	171.8	0	60,095
blocked n=6400, p=8	default	242.4	4.1032	127.8	0	240,096
	on	176.0	3.3269	157.6	936	948
	off	259.7	3.5886	146.1	0	240,095
ikj n=1600, p=1	default	6.8	0.3098	26.4	0	15,078
	on	3.1	0.2569	31.9	54	1,278
	off	6.5	0.3049	26.9	0	15,076

Transparent huge pages บน x86-64 หน้าปกติมีขนาด 4 KiB แต่ kernel ใช้หน้าขนาด 2 MiB ได้เมื่อช่วงหน่วยความจำ align และขอไว้ด้วย `madvise(MADV_HUGEPAGE)` [7] โปรแกรมจองเมทริกซ์ด้วย `posix_memalign` ให้ align ที่ 2 MiB เพื่อการนี้ ผลที่ชัดที่สุดคือจำนวน page fault ซึ่งลดลง 12–250 เท่า และตรวจสอบได้ด้วยการคำนวณ เมทริกซ์ขนาด $n^2 \times 8$ ไบต์ประกอบด้วยหน้า 2 MiB เต็มจำนวนหนึ่ง กับเศษปลายที่ยังเป็นหน้า 4 KiB ที่ $n = 1600$ เมทริกซ์ละ 20,480,000 ไบต์ = 9 หน้า 2 MiB บวกเศษ 392 หน้า 4 KiB สามเมทริกซ์จึงมี fault $3 \times (9 + 392) = 1{,}203$ ครั้ง บวก fault ที่ไม่เกี่ยวกับเมทริกซ์ 78 ครั้ง ($15{,}078 - 3 \times 5{,}000$) ได้ 1,281 ตรงกับที่วัดได้ทุกหลัก ที่ $n = 3200$ ($p = 8$) ได้ $3 \times (39 + 32) + 95 = 308$ และที่ $n = 6400$ ได้ $3 \times (156 + 128) + 96 = 948$ ตรงทุกหลักเช่นกัน AnonHugePages 54 MiB ที่ $n = 1600$ (ไม่ใช่ 58.6 MiB) ก็คือ 27 หน้า 2 MiB เต็มพอดี ผลนี้ยืนยันทั้งกลไก demand paging และการที่ kernel ใช้ huge page ได้เฉพาะช่วงที่ครอบคลุม 2 MiB ที่ align ครบทั้งหน้า เฟส init ซึ่งเป็นผู้แตะหน้าเกือบทั้งหมดเป็นครั้งแรกจึงเร็วขึ้น 2.2–2.4 เท่าที่ $n = 1600$–3200 (4 เท่าสำหรับ ikj ที่ $n = 3200$)

ผลต่อเวลา kernel ต้องตีความอย่างระมัดระวัง เพราะ default กับ off ควรเหมือนกันทุกประการ (ค่าของระบบเป็น madvise) แต่ที่ $p = 8$ ทั้งสองต่างกันถึง 12–14% (เช่น 4.10 เทียบกับ 3.59 s ที่ $n = 6400$) ระดับ noise ของการทดลองชุดนี้จึงอยู่ราว 15% เมื่อเทียบ on กับ off ที่ $p = 8$ blocked เร็วขึ้น 8% ที่ $n = 6400$ แต่ช้าลง 5% ที่ $n = 3200$ ซึ่งอยู่ในระดับ noise ทั้งคู่ ผลที่ชัดและสม่ำเสมอเกิดกับ ikj ที่ $n = 1600$ แบบ thread เดียว ซึ่งเร็วขึ้น 19% (26.9 → 31.9 GFLOP/s) คำอธิบายมาจาก TLB: L2 DTLB ของ Zen 5 มี 4,096 entry [4] ครอบคลุมหน่วยความจำได้ 16 MiB เมื่อใช้หน้า 4 KiB แต่ 8 GiB เมื่อใช้หน้า 2 MiB ikj อ่าน B ทั้งก้อนซ้ำทุกแถว i และ B ที่ $n = 1600$ มีขนาด 19.5 MiB เกิน 16 MiB เล็กน้อย การอ่านแต่ละรอบจึงเกิด TLB miss แทบทุกหน้า ขณะที่หน้า 2 MiB ใช้เพียง 10 entry ส่วน blocked ใช้ข้อมูลภายใน tile ซ้ำมาก จำนวนหน้าที่ต่างกันในช่วงเวลาสั้นๆ จึงน้อย และได้ประโยชน์จาก huge pages เพียง 1–4% ที่ thread เดียว

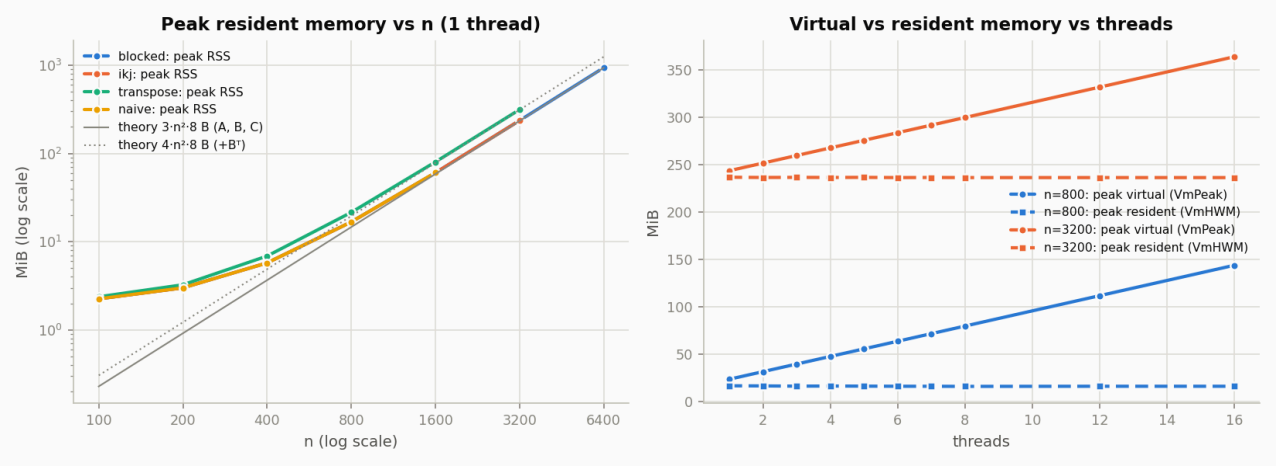


รูปที่ 8 ผลของ transparent huge pages ต่อเวลา init (บน) เวลา kernel (กลาง) และจำนวน minor page fault (ล่าง) ที่มา: results/plots/15_huge_pages.png

1.4.5 (e) ความเร็วและหน่วยความจำ

ตารางที่ 14 หน่วยความจำของ blocked (MiB) เทียบกับทฤษฎี (init แบบ serial)

n	threads	$3n^2 \cdot 8$	VmHWM (peak RSS)	VmPeak (virtual)	minor faults
100	1 / 8 / 16	0.23	2.25 / 2.12 / 2.12	3.1 / 59.2 / 123.2	131 / 157 / 163
800	1 / 8 / 16	14.65	16.75 / 16.12 / 16.25	23.6 / 79.7 / 143.7	3,828 / 3,845 / 3,862
1600	1 / 8 / 16	58.59	60.50 / 60.12 / 60.25	67.6 / 123.6 / 187.7	15,077 / 15,095 / 15,111
3200	1 / 8 / 16	234.38	236.50 / 236.12 / 236.12	243.4 / 299.4 / 363.4	60,077 / 60,094 / 60,112
6400	1 / 8	937.50	939.62 / 939.50	946.5 / 1,002.5	240,078 / 240,095



รูปที่ 9 ซ้าย: RSS สูงสุดตาม n ของทุกอัลกอริทึมเทียบกับ $3n^2 \cdot 8$ และ $4n^2 \cdot 8$ ไบต์; ขวา: VmPeak และ VmHWM ตามจำนวน thread ที่มา: results/plots/08_memory.png

RSS สูงสุด (VmHWM) ตรงกับ $3n^2 \cdot 8$ บวกประมาณ 2 MiB ในทุกขนาด (ตารางที่ 14) ส่วนเกิน 2 MiB คือโค้ดของโปรแกรม ไลบรารี (libc และ libgomp) stack และโครงสร้างของ runtime ซึ่งไม่ขึ้นกับ n และ transpose ใช้มากกว่าราว 33% ตามที่คาด (รูปที่ 9 ซ้าย) จำนวน minor page fault เท่ากับจำนวนหน้า 4 KiB ที่เมทริกซ์ใช้ เช่นที่ n = 3200 คือ 245,760,000 / 4,096 = 60,000 บวกอีก 77 แสดงว่าทุกหน้าของเมทริกซ์ถูกจัดหน้าจริงตอนที่โปรแกรมแตะครั้งแรก ไม่ใช่ตอน malloc [15]

VmPeak เพิ่มขึ้น 8.0 MiB ต่อ thread หนึ่งตัวพอดี ((299.4 – 243.4) / 7 = 8.00) แต่ RSS ไม่เพิ่มเลย และ minor fault เพิ่มเพียงราว 2 ครั้งต่อ thread เพราะ thread ใหม่แต่ละตัวได้ช่วงที่อยู่สำหรับ stack ขนาด 8 MiB (ขนาดเริ่มต้นของ pthread) แต่แตะจริงเพียงไม่กี่หน้า หน้าที่ไม่ได้แตะจึงไม่เคยถูกจัดหน้าจริงให้ ภาพเดียวกันนี้เห็นในส่วนที่ 2 ซึ่ง thread stack ขนาด 8 MiB มี RSS เพียง 72 KiB ข้อสรุปด้านความเร็วกับหน่วยความจำคือ การเพิ่ม thread ทำให้เร็วขึ้นโดยแทบไม่เสียหน่วยความจำจริง หน่วยความจำถูกกำหนดเกือบทั้งหมดโดยขนาดเมทริกซ์ ข้อแลกเปลี่ยนระหว่างความเร็วกับหน่วยความจำเกิดเฉพาะใน transpose ที่ใช้หน่วยความจำเพิ่ม 33% เพื่อแลกกับการอ่านแบบ stride 1 (ซึ่งเร็วกว่า ikj เฉพาะที่ 1 thread และ $n \le 200$) และใน huge pages ที่ kernel จัดหน่วยความจำทีละ 2 MiB ซึ่งอาจทำให้ RSS เกินที่ใช้จริงได้เกือบ 2 MiB ต่อช่วง แต่ได้ page fault ที่น้อยลงมาก

1.5 การวิเคราะห์ด้วย Amdahl's law

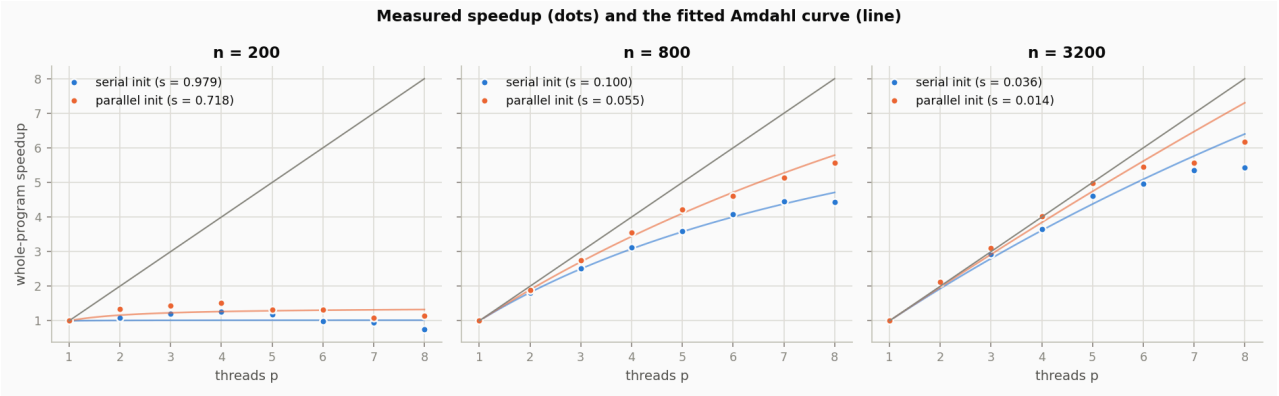
ถ้าส่วนของงานที่ต้องทำแบบลำดับคิดเป็นสัดส่วน s ของเวลาเมื่อใช้ thread เดียว speedup บน p thread มีขอบเขตตาม Amdahl's law [2]

$$S(p) = 1 / (s + (1 - s) / p) \quad \text{และ} \quad S(\infty) = 1 / s$$

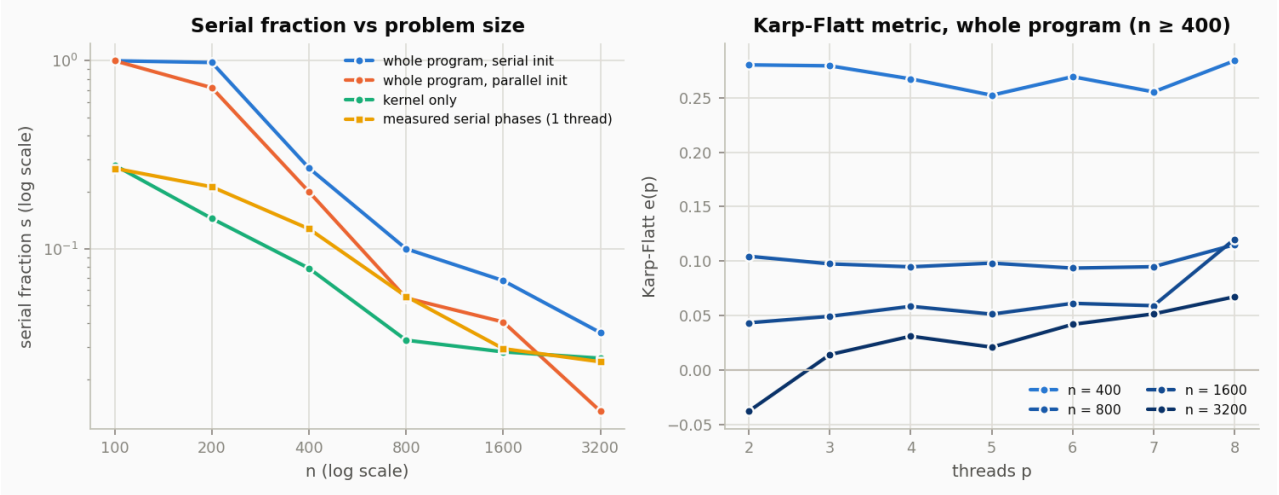
เราประมาณ s ได้สองวิธี วิธีแรก (s_{meas}) วัดตรงจากการรัน 1 thread ว่าเฟส serial (alloc + init + checksum) กินสัดส่วนเท่าใดของเวลารวม วิธีที่สอง (s_{fit}) หาค่า s ที่ทำให้สูตรตรงกับ speedup ที่วัดได้มากที่สุด โดยจัดสมการใหม่เป็น $1/S - 1/p = s \cdot (1 - 1/p)$ แล้วใช้ least squares ผ่านจุดกำเนิดกับ $x = 1 - 1/p$ และ $y = 1/S - 1/p$ สำหรับ p = 2–8 (เฉพาะ p ไม่เกินจำนวน CPU) นอกจากนี้ Karp–Flatt metric [17] $e(p) = (1/S(p) - 1/p) / (1 - 1/p)$ คือค่า s ที่คำนวณแยกทีละค่า p ถ้า e คงที่เมื่อ p เปลี่ยน การสูญเสียมาจากส่วน serial ที่มีขนาดคงที่จริง ถ้า e เพิ่มตาม p การสูญเสียมาจาก overhead ที่โตตามจำนวน thread ผลทั้งหมดอยู่ในตารางที่ 15 และรูปที่ 10–11

ตารางที่ 15 ผลการวิเคราะห์ Amdahl และ Karp–Flatt ของ blocked (ทั้งโปรแกรม เว้นแต่ระบุ)

init, n	s_{meas}	s_{fit}	R^2	S(8) วัดได้	$S(\infty) = 1/s_{fit}$	e(2)	e(4)	e(8)
serial, 100	0.267	1.000	−9.20	0.05	1.0	1.77	2.32	20.9
serial, 200	0.213	0.979	−0.09	0.75	1.0	0.833	0.727	1.379
serial, 400	0.127	0.268	0.979	2.68	3.7	0.280	0.267	0.284
serial, 800	0.056	0.100	0.985	4.43	10.0	0.104	0.095	0.115
serial, 1600	0.029	0.068	0.834	4.35	14.8	0.043	0.058	0.120
serial, 3200	0.025	0.036	0.879	5.44	28.0	−0.038	0.031	0.067
parallel, 800	0.056	0.055	0.990	5.56	18.3	0.063	0.043	0.063
parallel, 1600	0.028	0.041	0.979	5.78	24.6	0.023	0.035	0.055
parallel, 3200	0.033	0.014	0.823	6.17	73.4	−0.061	−0.001	0.042
kernel อย่างเดียว, 800	–	0.033	0.998	6.65	30.7	0.050	0.033	0.029
kernel อย่างเดียว, 1600	–	0.028	0.990	6.32	35.5	0.019	0.031	0.038
kernel อย่างเดียว, 3200	–	0.026	0.897	5.81	38.3	−0.040	0.023	0.054



รูปที่ 10 speedup ที่วัดได้ (จุด) กับเส้นโค้ง Amdahl จาก s_{fit} (เส้น) ที่ n = 200, 800 และ 3200 สำหรับ init แบบ serial และแบบขนาน ที่มา: results/plots/05_amdahl_fit.png



รูปที่ 11 ซ้าย: สัดส่วน serial ตาม n จากการ fit และจากการวัดตรง; ขวา: Karp–Flatt metric ของทั้งโปรแกรม (n ≥ 400) ที่มา: results/plots/06_serial_fraction.png

ขนาดที่ Amdahl ใช้ไม่ได้ ที่ n = 100 และ 200 การ fit ให้ $s \approx 1$ และ R^2 ติดลบ เพราะ speedup ต่ำกว่า 1 ซึ่ง Amdahl's law อธิบายไม่ได้เลย (สูตรให้ $S \geq 1$ เสมอ) เวลาที่เพิ่มตาม p มาจากการสร้าง thread ซึ่งเป็น overhead ที่โตตามจำนวน thread ไม่ใช่ส่วน serial ที่คงที่ ค่า e(8) = 20.9 ที่ n = 100 บอกเรื่องเดียวกัน

s_{fit} มากกว่า s_{meas} (init แบบ serial) s_{meas} นับเฉพาะโค้ดที่เป็น serial จริง ส่วน s_{fit} คือ serial fraction ที่มีผลจริง ซึ่งรวมทุกอย่างที่ไม่เร็วขึ้นตาม p ได้แก่ เวลาสร้าง thread, load imbalance, การรอที่ barrier, การแย่ง memory bandwidth และความถี่ที่ลดลงเมื่อหลายคอร์ทำงาน ผลต่าง $s_{fit} - s_{meas}$ จึงเป็นตัวชี้วัด overhead ของการขนาน ที่ n = 400 ผลต่างใหญ่มาก (0.268 เทียบกับ 0.127) เพราะ kernel ใช้ 3.44 ms ที่ thread เดียวและ 0.60 ms ที่ 8 thread ขณะที่การสร้างทีม 8 thread ใช้ 0.42 ms ซึ่งมีขนาดเทียบเคียงกัน ค่า Karp–Flatt ที่ n = 400 และ 800 เกือบคงที่ตาม p (0.27–0.28 และ 0.095–0.115) แปลว่าการสูญเสียที่ขนาดเหล่านี้มาจากส่วนที่มีขนาดคงที่ คือเฟส serial กับเวลา spawn ส่วนที่ n = 1600 และ 3200 ค่า e เพิ่มตาม p ชัดเจน (0.043 → 0.120 และ −0.038 → 0.067) แปลว่ามี overhead ที่โตตามจำนวน thread และ e(8) สูงกว่า s_{meas} 2.7–4.1 เท่า แม้แต่ kernel ล้วนซึ่งไม่มีโค้ด serial เลยก็ยังมี e(8) = 0.038–0.054 สิ่งที่ขวางไม่ให้ speedup ถึง 8 ที่ n ใหญ่จึงไม่ใช่โค้ด serial เป็นหลัก แต่เป็น overhead ของการใช้ครบ 8 คอร์ ได้แก่ bandwidth ความถี่ และการแย่ง CPU กับ host ที่วิเคราะห์ในหัวข้อ 1.4.1 ถ้ารวมสองผลเป็น $S \approx 1 / (s_{meas} + (1 - s_{meas})/K)$ โดย K = 5.81 คือ speedup ของ kernel ที่ n = 3200 จะได้ $1 / (0.025 + 0.975/5.81) \approx 5.19$ ใกล้กับ 5.44 ที่วัดได้ (ส่วนที่วัดได้สูงกว่าเล็กน้อยมาจาก T(1) ที่ยาวเกินจริงตามหัวข้อ 1.4.1)

สัดส่วน serial ลดลงเมื่อ n โต เฟส serial (init และ checksum) มีงาน $O(n^2)$ ขณะที่ kernel มีงาน $O(n^3)$ ดังนั้น $s \approx c/n$ สำหรับค่าคงที่ c บางค่า ข้อมูลยืนยันแนวโน้มนี้ $s_{meas} \times n$ = 51.0, 44.6 และ 46.9 ที่ n = 400, 800 และ 1600 ซึ่งเกือบคงที่ ส่วนที่ n = 3200 ได้ 80 สูงกว่าแนวโน้ม ซึ่งอธิบายได้ด้วยเฟส init ของการรัน p = 1 ที่ช้ากว่าปกติราวสองเท่าดังที่พบในหัวข้อ 1.4.1 ข้อสังเกตนี้คือแนวคิดของ Gustafson's law [18] ถ้าเพิ่มขนาดปัญหาไปพร้อมกับจำนวนคอร์ ส่วน serial จะเล็กลงเรื่อยๆ และ scaled speedup $S = p - s(p - 1)$ จะเข้าใกล้ p เช่นที่ n = 3200 ได้ $8 - 0.025 \times 7 = 7.83$ เพดานตาม Amdahl's law จึงขึ้นกับว่ามองปัญหาขนาดคงที่ (Amdahl) หรือปัญหาที่โตตามทรัพยากร (Gustafson)

ผลของ init แบบขนาน เฟส init เป็นส่วน serial ที่ใหญ่ที่สุดและทำขนานได้ เพราะแต่ละสมาชิกคำนวณอิสระจากตำแหน่งของตัวเอง เมื่อใช้ `-p` ค่า s_{fit} ลดลงทุกขนาด ที่ n = 800 จาก 0.100 เหลือ 0.055 ที่ n = 1600 จาก 0.068 เหลือ 0.041 และที่ n = 3200 จาก 0.036 เหลือ 0.014 speedup ที่ 8 thread ของทั้งโปรแกรมเพิ่มจาก 5.44 เป็น 6.17 เท่า และเพดาน 1/s เพิ่มจาก 28 เป็น 73 เท่า ข้อดีเพิ่มเติมคือ page fault ถูกกระจายไปหลาย thread แทนที่จะเกิดใน thread เดียว (ที่ n = 3200 init ลดจาก 24 ms เหลือราว 7–9 ms ที่ 8 thread)

> **คำตอบของโจทย์: ส่วนขนานต่อส่วน serial ประมาณเท่าใด** คำตอบขึ้นกับ n และวิธีนับ (ตารางที่ 16) ถ้านับเฉพาะโค้ดที่เป็น serial จริง (s_{meas}) อัตราส่วนเป็นประมาณ 7 : 1 ที่ n = 400, 17 : 1 ที่ n = 800, 33 : 1 ที่ n = 1600 และ 39 : 1 ที่ n = 3200 ถ้านับ serial ที่มีผลจริงจาก speedup ที่วัดได้ (s_{fit}) ซึ่งรวม overhead ไว้ด้วย อัตราส่วนเป็นประมาณ 2.7 : 1, 9 : 1, 14 : 1 และ 27 : 1 ตามลำดับ และเพิ่มเป็นประมาณ 72 : 1 ที่ n = 3200 เมื่อ init แบบขนาน สรุปคือที่ขนาดใหญ่ที่สุดที่ทดลอง ส่วนขนานคิดเป็นราว 96.4–97.5% ของงาน และสัดส่วนนี้เข้าใกล้ 100% เมื่อ n โตขึ้น เพราะส่วนขนานโตเป็น n^3 ขณะที่ส่วน serial โตเป็น n^2 ส่วนที่ $n \leq 200$ คำถามนี้ไม่มีความหมายนัก เพราะเวลาถูกครอบงำด้วย overhead ของการสร้าง thread ไม่ใช่ทั้งส่วนขนานหรือส่วน serial

ตารางที่ 16 อัตราส่วนส่วนขนานต่อส่วน serial ของ blocked

n	ขนาน : serial (s_{meas})	ส่วนขนาน (s_{meas})	ขนาน : serial (s_{fit})	ส่วนขนาน (s_{fit})	S(8) ตามสูตรจาก s_{meas}
400	6.8 : 1	87.3%	2.7 : 1	73.2%	4.23
800	17.0 : 1	94.4%	9.0 : 1	90.0%	5.76
1600	33.1 : 1	97.1%	13.8 : 1	93.2%	6.64
3200	39.0 : 1	97.5%	27.0 : 1	96.4%	6.81
3200, init ขนาน	–	–	72.5 : 1	98.6%	–

1.6 ข้อจำกัดของการทดลอง

การทดลองทั้งหมดทำบนเครื่องเดียวภายใน WSL2 ซึ่ง host ยังมีโปรแกรมอื่นทำงาน ความแปรปรวนจึงสูงเป็นพิเศษที่ p = 8 และที่ n เล็ก (coefficient of variation สูงสุด 704%) และจากการเทียบ THP แบบ default กับ off ซึ่งควรเท่ากัน ระดับ noise ของเวลา kernel ที่ 8 thread อยู่ราว 15% และค่าตั้งเดียวกันที่วัดในต่างชุดการทดลองยังต่างกันได้ถึง 30% (unset ที่ n = 3200, p = 8 ใช้ 368 ms ในชุด scaling แต่ 482 ms ในชุด osbind) ผลต่างที่เล็กกว่านี้จึงไม่ควรตีความ ค่ามัธยฐานจาก 3–5 รอบลดผลของค่าผิดปกติได้แต่ไม่ได้กำจัดทิ้ง ลำดับการรันของ bench.py (เรียงตามจำนวน thread) ทำให้การรัน p = 1 ที่ n ≥ 3200 เสียเปรียบในเฟส init ซึ่งควรแก้ด้วยการสลับลำดับแบบสุ่มหรือรันอุ่นเครื่องก่อน และเมื่อคูณเพียงรอบเดียว เวลา kernel รวม page fault ของ C ไว้ด้วย ความถี่ของ CPU เปลี่ยนตามจำนวนคอร์ที่ทำงานและอุณหภูมิ ซึ่งไม่ได้ควบคุมหรือบันทึกไว้ การวิเคราะห์ cache และ TLB ในรายงานนี้เป็นการคำนวณจากข้อมูลจำเพาะของ CPU ไม่ได้วัดจำนวน cache miss หรือ TLB miss โดยตรงด้วย hardware performance counter ซึ่งมักเข้าถึงไม่ได้ใน WSL2 สุดท้าย ผลเชิงตัวเลขเป็นของ gcc 13.3, libgomp และ CPU รุ่นนี้ บน compiler หรือ CPU อื่นตัวเลขอาจต่างไป แม้แนวโน้มเชิงคุณภาพควรเหมือนเดิม

ส่วนที่ 2 การแสดงภาพ Memory Map ของโปรเซส

2.1 เป้าหมายและข้อมูลขาเข้า

เครื่องมือ `mmap.py` แสดงภาพ address space ของโปรเซสบน Linux โดยเลือกใช้ข้อมูลขาเข้าแบบ memory map ที่ OS จัดให้ ตามที่โจทย์ระบุว่าเป็นทางเลือกที่ง่ายที่สุด เพราะ kernel เก็บรายการ virtual memory area (VMA) ของทุกโปรเซสไว้อยู่แล้วและเปิดเผยผ่านระบบไฟล์ /proc [13], [19] เครื่องมือแสดงข้อมูลเป็นสามระดับ และแต่ละระดับมาจากไฟล์ต่างกันดังตารางที่ 17

ตารางที่ 17 แหล่งข้อมูลใน /proc ที่ mmap.py ใช้

ระดับ	ไฟล์	ข้อมูลที่ได้
โปรเซส	/proc/PID/status	VmSize, VmRSS (แยกเป็น RssAnon, RssFile, RssShmem), VmSwap, จำนวน thread, ชื่อและ command line
region (VMA)	/proc/PID/maps และ /proc/PID/smaps	ช่วงที่อยู่ สิทธิ์ (r, w, x และ private/shared) offset และไฟล์ที่ map; smaps เพิ่ม Rss, Pss, Swap, AnonHugePages และ VmFlags ของแต่ละ region
page	/proc/PID/pagemap	64 บิตต่อหน้า: บิต 63 อยู่ใน RAM, บิต 62 ถูก swap, บิต 61 เป็นหน้าของไฟล์หรือ shared anonymous, บิต 56 map แบบ exclusive [20]
thread	/proc/PID/task/TID/syscall	stack pointer ของ thread ที่หยุดอยู่ใน system call ใช้ระบุว่า region ใดเป็น stack ของ thread ใด

ไฟล์เหล่านี้อ่านได้เฉพาะโปรเซสของผู้ใช้คนเดียวกัน (หรือเมื่อเป็น root) และเครื่องมืออ่านอย่างเดียว ไม่เขียนหรือส่ง signal ไปยังโปรเซสอื่น ยกเว้นโปรเซสลูกที่ตัวเองเปิดด้วย `--run` ซึ่งจะถูกหยุดเมื่อเครื่องมือจบการทำงาน

2.2 การออกแบบและการพัฒนา

mmap.py เป็นโปรแกรม Python 3 ไฟล์เดียวที่ใช้เพียง standard library จึงรันได้ทันทีบน Linux โดยไม่ต้องติดตั้งอะไรเพิ่ม การทำงานแบ่งเป็นห้าขั้น ขั้นแรกอ่าน smaps (หรือ maps ถ้าไม่มีสิทธิ์อ่าน smaps) และ status ขั้นที่สองแยกวิเคราะห์บรรทัดหัวของแต่ละ region ด้วย regular expression แล้วเก็บฟิลด์ของ smaps ที่ตามมา ขั้นที่สามจำแนกแต่ละ region เข้าหมวดใดหมวดหนึ่งในแปดหมวด (ตารางที่ 18) พร้อมคำอธิบาย ขั้นที่สี่อ่าน pagemap เพื่อคำนวณว่าแต่ละช่วงย่อยของ region (48 ช่วงต่อ region) มีหน้าที่อยู่ใน RAM กี่เปอร์เซ็นต์ ซึ่งใช้วาดว่าส่วนไหนของ region ถูกใช้จริง ขั้นสุดท้ายแสดงผลใน terminal หรือเขียนเป็น HTML และ JSON ในโหมดติดตามต่อเนื่อง เครื่องมือเปรียบเทียบ snapshot ที่อยู่ติดกันแล้วสร้างรายการเหตุการณ์ ได้แก่ region ใหม่ region ที่เปลี่ยนขนาด region ที่ถูกยกเลิก และ RSS ที่เปลี่ยนเกิน 1 MiB โดยจับคู่ region ที่โตลงล่าง (stack) จากที่อยู่ปลายแทนที่อยู่ต้น

ตารางที่ 18 กฎการจำแนก region (ฟังก์ชัน `classify` และ `classify_one`)

หมวด	กฎการจำแนก
program (executable)	ไฟล์ที่ map ตรงกับ /proc/PID/exe แยกเป็น .text (r-x), .rodata/RELRO (r--) และ .data (rw-); region ไม่มีชื่อที่ต่อจาก data ทันทีคือ .bss โดยคำนวณจุดสิ้นสุดของ .bss จาก program header ของไฟล์ ELF (PT_LOAD: p_vaddr + p_memsz) ใช้กฎเดียวกันกับ .bss ของไลบรารี
heap (brk)	region ชื่อ [heap]
stack	[stack] คือ stack ของ main thread; region ที่มี stack pointer ของ thread อื่นอยู่ข้างในคือ thread stack (ถ้าใหญ่เกิน 16 MiB ถือว่าถูก kernel รวมกับ region อื่น); ถ้าอ่าน stack pointer ไม่ได้ ใช้ heuristic ว่าเป็น region rw ที่มี guard page (---p ไม่เกิน 64 KiB) อยู่ข้างล่าง
anonymous mmap	malloc arena ของ thread อื่น (คู่ rw-p กับ ---p ที่ขนาดรวมเป็นพหุคูณของ 64 MiB) และหน่วยความจำจาก mmap() หรือ malloc() ขนาดใหญ่ทั่วไป
shared library	ไฟล์ที่ชื่อมี .so
mapped file	ไฟล์อื่นที่ถูก map (ระบุเพิ่มว่าไฟล์ถูกลบแล้วหรือเป็น MAP_SHARED)
shared memory	/dev/zero (จาก MAP_SHARED | MAP_ANONYMOUS), /dev/shm, memfd, SysV หรือ region ที่มีสิทธิ์ s
guard / reserved / kernel	region ---p (guard page หรือที่อยู่ที่จองไว้), [vdso], [vvar] และ [vsyscall]

2.3 ความสามารถและวิธีใช้งาน

ตารางที่ 19 คำสั่งหลักของ mmap.py

คำสั่ง	ผลลัพธ์
`python3 mmap.py --list`	รายการโปรเซสที่ตรวจดูได้ พร้อม RSS และขนาด virtual
`python3 mmap.py PID` (หรือ `self`)	snapshot หนึ่งครั้งใน terminal; `--compact` รวม region ของไฟล์เดียวกันเป็นแถวเดียว
`python3 mmap.py PID --watch 1`	มุมมองแบบ real time อัปเดตทุก 1 วินาที พร้อมเหตุการณ์ที่เปลี่ยน
`python3 mmap.py PID --html map.html`	รายงาน HTML ไฟล์เดียวที่โต้ตอบได้
`python3 mmap.py --run "./demo -s 1" --record 40 --html demo.html`	เปิดโปรแกรมแล้วบันทึก timeline เป็น HTML
`python3 mmap.py PID --pages 12`	แผนที่ทีละหน้าของ region #12 จาก pagemap (หน้าใดอยู่ใน RAM หรือถูก swap)
`python3 mmap.py --file saved_smaps.txt`	วาดจากไฟล์ maps หรือ smaps ที่บันทึกไว้

มุมมอง terminal (รูปที่ 12) ประกอบด้วยส่วนหัวที่สรุปค่าระดับโปรเซส แถบ low...high ที่ย่อทั้ง address space ลงในบรรทัดเดียว (ความกว้างของแต่ละส่วนแปรตาม log ของขนาด และลวดลายบอกสัดส่วนที่อยู่ใน RAM) ตารางสรุป RSS และ PSS ตามหมวด รายการเหตุการณ์ล่าสุด และตาราง region เรียงจากที่อยู่สูงลงต่ำเหมือนภาพในตำรา แถวที่มีเครื่องหมาย + คือ region ใหม่ และ * คือ region ที่ขนาดหรือ RSS เปลี่ยนจาก snapshot ก่อนหน้า ตารางสรุปตามหมวดยังแสดงความต่างระหว่าง RSS กับ PSS ได้ชัด เช่น shared library มี RSS 1.7 MiB แต่ PSS เพียง 136 KiB เพราะหน้าของ libc ที่ใช้ร่วมกับโปรเซสอื่นถูกหารตามจำนวนโปรเซสที่ใช้ร่วม รายงาน HTML (รูปที่ 13 และ 17) เป็นไฟล์เดียวที่เปิดด้วย browser ได้ ประกอบด้วย timeline ของ RSS แยก

ตามหมวดที่เลื่อนเลือก snapshot ได้ แผนที่ address space ที่ความสูงของกล่องแปรตาม log ของขนาดและแถบสีภายในกล่องแสดงสัดส่วนหน้าที่อยู่ใน RAM ตามตำแหน่ง (จาก pagemap) ตารางทุก region ที่กรองและเรียงได้ และ tooltip รายละเอียดของแต่ละ region



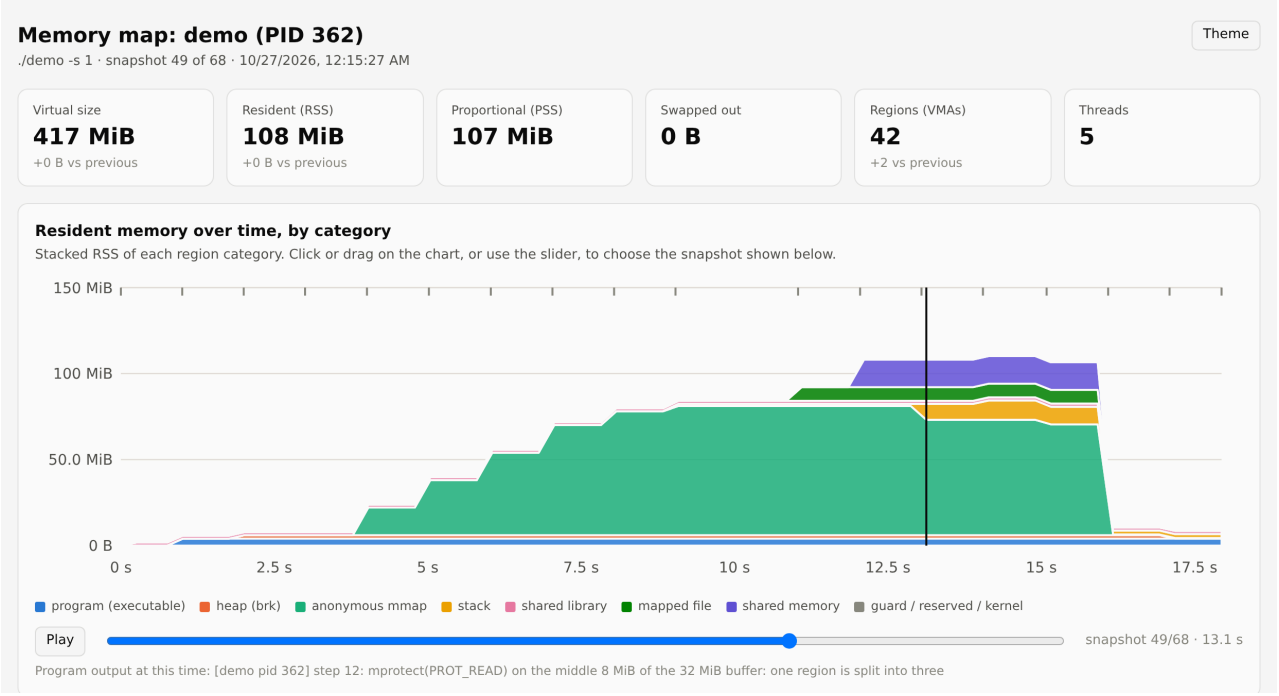
รูปที่ 12 มุมมอง terminal ของ mmap.py ที่ snapshot 37 จาก 68 (หลังสร้าง 4 thread) ภาพนี้สร้างโดยฟังก์ชันแสดงผลของ mmap.py เองจากข้อมูล snapshot ที่บันทึกไว้ใน demo_timeline.html

2.4 กรณีศึกษา: โปรแกรม demo

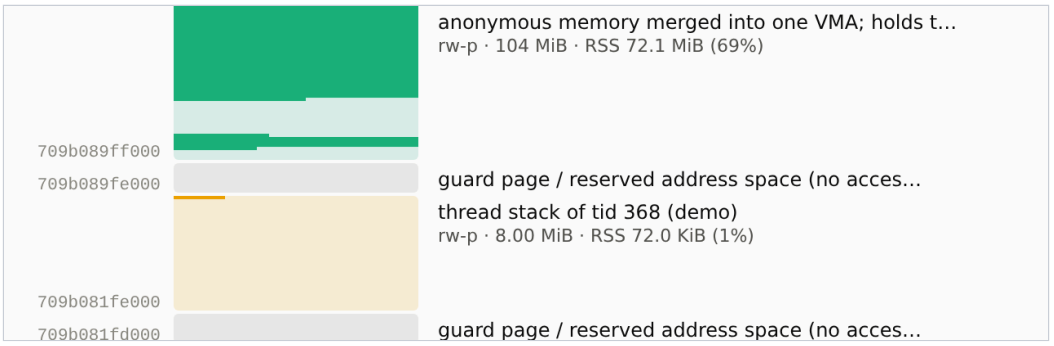
`demo.c` เป็นโปรแกรมเป้าหมายที่เปลี่ยน memory map ทีละขั้น ขั้นละ 1 วินาที (รันด้วย `./demo -s 1`) ข้อมูลในหัวข้อนี้มาจากการบันทึกด้วยโหมด `--record` ของ mmap.py บน WSL2 ได้ 68 snapshot ห่างกันประมาณ 0.25 วินาทีตลอด 18 วินาที (PID 362) ตารางที่ 20 สรุปค่าระดับโปรเซสเมื่อจบแต่ละขั้น และรูปที่ 15 แสดง VmSize และ VmRSS ตามเวลา

ตารางที่ 20 ค่าระดับโปรเซสเมื่อจบแต่ละขั้นของ demo (MiB)

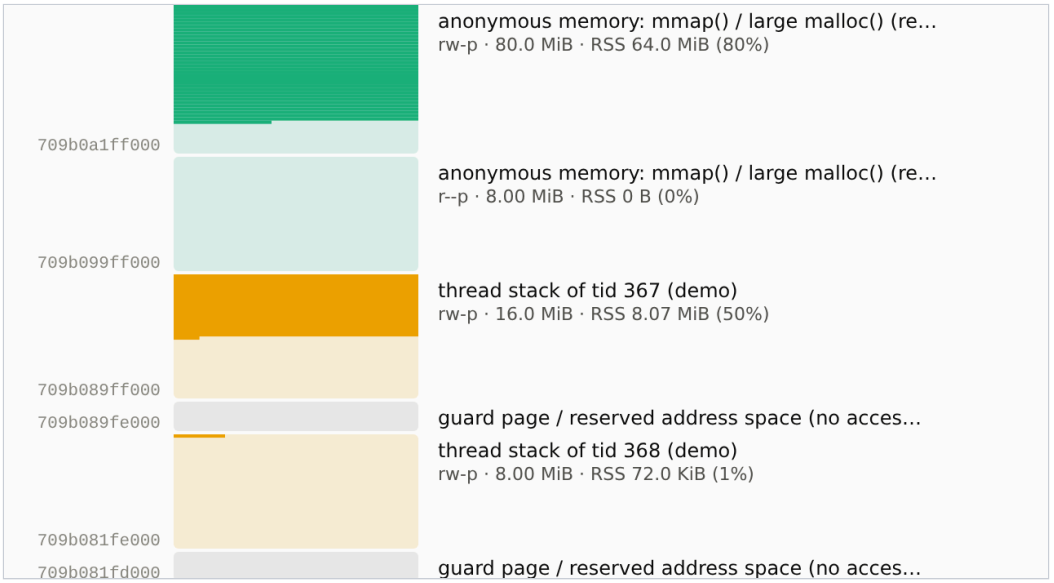
ขั้น	การกระทำ	VmSize	VmRSS	region	สิ่งที่เห็นใน memory map
0	เริ่มโปรแกรม	6.6	1.6	23	โค้ด, .rodata, .data, .bss ของ demo, libc, ld-linux, [heap] 132 KiB, [stack], vdso/vvar
1	เขียน .bss 4 MiB	6.6	5.6	23	RSS ของ .bss 0 → 4 MiB, VmSize ไม่เปลี่ยน
2	malloc(512) × 4,000	8.6	7.8	23	[heap] ขยายด้วย brk จาก 132 KiB เป็น 2,112 KiB
3	malloc(64 MiB)	72.6	7.8	24	region ใหม่ 64 MiB + 4 KiB (mmap) แต่ RSS ไม่เปลี่ยน
4–7	เขียน block ทีละ 16 MiB	72.6	23.8 → 71.8	24	RSS เพิ่มขั้นละ 16.0 MiB พอดี
8	mmap 32 MiB เขียน 8 MiB	104.6	79.8	24	ไม่มี region ใหม่: ถูกรวมกับ block เดิมเป็น 96 MiB
9	สร้าง 4 thread	392.6	83.8	38	stack 8 MiB + guard 4 KiB และ malloc arena 64 MiB ต่อ thread; region รวมโตเป็น 104 MiB
10	mmap ไฟล์ 8 MiB ที่ถูกลบแล้ว	400.6	91.8	39	region ไฟล์ (deleted), RssFile 1.6 → 9.6 MiB
11	mmap MAP_SHARED | MAP_ANONYMOUS 16 MiB	416.6	107.8	40	/dev/zero (deleted) สิทธิ์ rw-s, RssShmem 16 MiB
12	mprotect 8 MiB กลางของ buffer	416.6	107.8	42	region 104 MiB แยกเป็น 16 + 8 (r--) + 80 MiB
13	recursion 128 ชั้น ชั้นละ 16 KiB	418.5	109.8	42	[stack] โตลงจาก 132 KiB เป็น 2,080 KiB
14	join ทุก thread	418.5	106.5	42	thread 5 → 1, VMA ของ stack และ arena ยังอยู่ แต่ RSS ลด
15	free(64 MiB) และ munmap สามครั้ง	298.5	10.5	38	4 region หายไป, VmSize ลด 120 MiB
16	free block เล็กทั้งหมด + malloc_trim(0)	298.5	8.5	38	[heap] 2,112 → 2,072 KiB แต่ RSS ของ heap 2,068 → 20 KiB



รูปที่ 13 รายงาน HTML ของ mmap.py ที่ snapshot 49 (ขั้นที่ 12, mprotect): ค่าสรุประดับโปรเซสและ timeline ของ RSS แยกตามหมวด

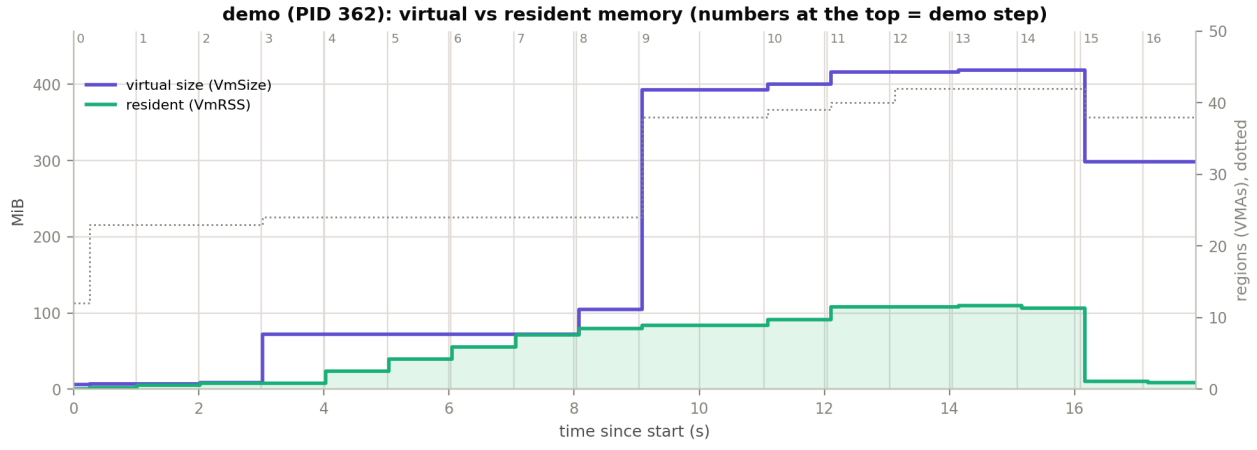


(ก) snapshot 37 หลังสร้าง thread: stack ของ tid 367, buffer 32 MiB และ block 64 MiB ถูกรวมเป็น region เดียว 104 MiB



(ข) snapshot 49 หลัง mprotect: region เดิมแยกเป็น 80 MiB (rw-p), 8 MiB (r--p) และ 16 MiB (rw-p)

รูปที่ 14 ส่วนของแผนที่ address space ในรายงาน HTML ช่วงใต้ libc ลงไปถึง stack ของ tid 368 (ที่อยู่สูงอยู่ด้านบน) สีเขียวคือ anonymous mmap สีเหลืองคือ stack และสีเทาคือ guard page ส่วนที่ระบายสีทึบในแต่ละกล่องคือหน้าที่อยู่ใน RAM ตามตำแหน่ง (ส่วนล่างของกล่องคือที่อยู่ต่ำ) VmSize และ RSS ของทั้งสองภาพเท่ากัน



รูปที่ 15 VmSize และ VmRSS ของ demo ตามเวลา จาก 68 snapshot ใน demo_timeline.html (ตัวเลขด้านบนคือหมายเลขขั้น เส้นประคือจำนวน region)

2.5 ข้อสังเกตและการวิเคราะห์

Demand paging: virtual ไม่เท่ากับ resident

ขั้นที่ 3 malloc(64 MiB) ทำให้ VmSize เพิ่ม 64 MiB ทันทีแต่ RSS ไม่เปลี่ยนเลย เพราะ kernel เพียงสร้าง VMA ใหม่ ยังไม่จัดหน้าจริงให้ ในขั้นที่ 4–7 ทุกครั้งที่โปรแกรมเขียนหนึ่งไบต์ต่อหน้าใน 16 MiB ถัดไป RSS เพิ่ม 16.0 MiB พอดี เพราะการแตะหน้าครั้งแรกทำให้เกิด page fault และ kernel จัดหน้าว่างที่เติมศูนย์แล้วให้ทีละหน้า [15] ขั้นที่ 1 แสดงสิ่งเดียวกันกับ .bss อาร์เรย์ 4 MiB อยู่ใน address space ตั้งแต่เริ่มโปรแกรมแต่มี RSS เป็นศูนย์ จนกว่าจะถูกเขียน รูปที่ 15 แสดงช่องว่างระหว่าง VmSize กับ VmRSS ซึ่งกว้างที่สุดหลังสร้าง thread (393 MiB เทียบกับ 84 MiB)

malloc เลือกใช้ brk หรือ mmap ตามขนาด

block ขนาด 512 ไบต์ 4,000 ก้อน (รวมส่วนหัวของ malloc ประมาณ 2,062 KiB) ถูกจัดจาก [heap] ซึ่ง glibc ขยายด้วย brk() จาก 132 KiB เป็น 2,112 KiB ส่วน block 64 MiB ใหญ่กว่า M_MMAP_THRESHOLD (ค่าเริ่มต้น 128 KiB) glibc จึงขอด้วย mmap() เป็น region ของตัวเอง ขนาด 64 MiB + 4 KiB ซึ่งส่วนเกินคือส่วนหัวของ chunk ที่ถูกปัดขึ้นเป็นหนึ่งหน้า [21], [22] ข้อดีของการแยกแบบนี้คือ block ใหญ่คืนให้ OS ได้ทันทีเมื่อ free() (ขั้นที่ 15) โดยไม่ติดปัญหา fragmentation ของ heap [23]

Linux รวม VMA ที่อยู่ติดกัน

ขั้นที่ 8 ควรเกิด region ใหม่ขนาด 32 MiB แต่จำนวน region ยังเป็น 24 เท่าเดิม และ region ของ block 64 MiB โตลงล่างเป็น 96 MiB แทน เหตุผลคือ mmap() วาง mapping ใหม่ไว้ที่อยู่ต่ำกว่า mapping ก่อนหน้าแบบติดกันพอดี และเมื่อ VMA สองตัวที่ติดกันเป็น anonymous private ที่มีสิทธิ์และ flag เหมือนกัน kernel จะรวมเป็น VMA เดียวเพื่อประหยัดโครงสร้างข้อมูล [24] demo พยายามตั้งชื่อ region ด้วย prctl(PR_SET_VMA_ANON_NAME) ซึ่งจะทำให้ VMA ต่างกันและไม่ถูกรวม แต่ prctl() ล้มเหลวบนเครื่องที่ทดลอง ความสามารถนี้ต้องใช้ Linux 5.17 ขึ้นไปที่คอมไพล์ด้วย CONFIG_ANON_VMA_NAME [21] แม้ kernel ของ WSL2 ที่ใช้ (6.6.87.2) จะใหม่กว่า 5.17 จึงสรุปได้ว่า kernel นี้ไม่ได้เปิดตัวเลือกดังกล่าว ในขั้นที่ 9 stack ของ thread แรก (8 MiB) ถูกวางต่อลงมาอีกและถูกรวมเป็น region เดียวขนาด 104 MiB ซึ่งประกอบด้วย stack 8 MiB, buffer 32 MiB และ block 64 MiB + 4 KiB เครื่องมือจึงรายงานว่าเป็น anonymous memory ที่ถูกรวมเป็น VMA เดียวและมี stack ของ tid 367 อยู่ข้างใน (หาได้จาก stack pointer) แทนที่จะเรียกทั้ง 104 MiB ว่า stack

ขั้นที่ 12 แสดงทิศทางกลับกัน mprotect(PROT_READ) กับ 8 MiB กลางของ buffer ทำให้ส่วนนั้นมีสิทธิ์ต่างจากเพื่อนบ้าน kernel จึงต้องแยก VMA 104 MiB เป็นสามส่วน คือ 16 MiB (stack 8 MiB กับส่วนแรกของ buffer 8 MiB), 8 MiB ที่เป็น r-- และ 80 MiB (ส่วนท้ายของ buffer 16 MiB กับ block 64 MiB) จำนวน region เพิ่มจาก 40 เป็น 42 โดยที่ VmSize และ RSS ไม่เปลี่ยนเลย (รูปที่ 14) ผลข้างเคียงที่เห็นใน timeline ของรูปที่ 13 คือพื้นที่สีเขียว (anonymous) ลดลงและสีเหลือง (stack) เพิ่มขึ้นราว 8 MiB ที่ขั้นนี้ ไม่ใช่เพราะหน่วยความจำย้ายที่ แต่เพราะ region 16 MiB ที่แยกออกมามีขนาดไม่เกินเกณฑ์ 16 MiB เครื่องมือจึงจัดทั้ง region (รวม 8 MiB ของ buffer) เป็น thread stack ข้อนี้แสดงข้อจำกัดของการจำแนกในระดับ VMA เมื่อ VMA หนึ่งมีข้อมูลหลายชนิดปนกัน

Thread: stack, guard page และ malloc arena

การสร้าง 4 thread ในขั้นที่ 9 เพิ่ม VmSize 288 MiB (104.6 → 392.6 MiB) เท่ากับ 4 × (stack 8 MiB + guard page 4 KiB) + 4 × malloc arena 64 MiB หรือราว 32 + 256 MiB พอดี แต่ RSS เพิ่มเพียง 4 MiB thread แต่ละตัวได้ stack 8 MiB ที่มี guard page สิทธิ์ ---p ขนาด 4 KiB อยู่ใต้สุด ถ้า stack ล้นลงมาชน guard page จะเกิด segmentation fault แทนที่จะเขียนทับข้อมูลอื่นอย่างเงียบๆ stack แต่ละตัวมี RSS เพียง 72 KiB (อาร์เรย์ local 64 KiB ของฟังก์ชัน worker บวกเฟรมของฟังก์ชัน) และแผนที่ใน HTML แสดงว่าหน้าที่อยู่ใน RAM อยู่เฉพาะปลายบน (ที่อยู่สูง) ของ stack เพราะ stack บน x86-64 โตจากที่อยู่สูงลงต่ำ

เมื่อ thread เรียก malloc() ขนาด 64 KiB ซึ่งต่ำกว่า mmap threshold glibc ให้แต่ละ thread ใช้ arena ของตัวเองเพื่อลดการแย่ง lock กับ main arena [22] arena หนึ่งตัวจองที่อยู่ไว้ 64 MiB แต่เปิดให้ใช้ (rw-p) เพียงส่วนต้นตามที่ใช้จริง ซึ่งในที่นี้คือ 16 × 64 KiB ≈ 1 MiB ส่วนที่เหลือราว 63 MiB เป็น ---p ที่ยังใช้ไม่ได้ ตรงกับคู่ region 1.0 MiB และ 63.0 MiB ในรูปที่ 12 arena ตัวที่สี่น่าสนใจ ส่วน 1 MiB แรกของมันอยู่ติดกับ stack ของ thread 369 และมีสิทธิ์เหมือนกัน kernel จึงรวมเป็น region เดียวขนาด 9 MiB ส่วนที่เหลือ 63 MiB จึงไม่มีคู่ rw-p ให้ heuristic จับได้และถูกจำแนกเป็น guard/reserved แทนที่จะเป็น malloc arena นี่เป็นตัวอย่างข้อจำกัดของ heuristic ที่ต้องเดาจากรูปแบบของ VMA ข้างเคียง

หลัง join ทุก thread (ขั้นที่ 14) จำนวน thread ลดจาก 5 เหลือ 1 แต่ VMA ของ stack และ arena ยังอยู่ครบ (VmSize ไม่เปลี่ยน) เพราะ glibc เก็บ stack ของ thread ที่จบแล้วไว้ใช้ซ้ำกับ thread ถัดไปโดยไม่ต้อง mmap ใหม่ และ arena คงอยู่ตลอดอายุโปรเซส สิ่งที่เปลี่ยนคือ RSS ลดลง 3.3 MiB RSS ของแต่ละ arena ลดจาก 1.0 MiB เหลือ 132 KiB หลัง worker free() block ของตัวเอง และ RSS ของ stack แต่ละตัวลดจาก 72 KiB เหลือ 24 KiB ซึ่งสอดคล้องกับการที่ glibc แจ้ง kernel ให้ทิ้งหน้าของ stack ที่ไม่ใช้แล้ว (madvise) ตอน thread จบ โดยยังคง VMA ไว้ ข้อสังเกตนี้ย้ำว่าการคืนหน่วยความจำให้ OS ทำได้สองระดับ คือคืนทั้งช่วงที่อยู่ (munmap ซึ่ง VMA หายไป) หรือคืนเฉพาะหน้าจริง (madvise ซึ่ง VMA ยังอยู่แต่ RSS ลด)

munmap และ malloc_trim

ขั้นที่ 15 free() ของ block 64 MiB เรียก munmap() ภายใน และ munmap() ของ buffer ไฟล์ และ shared memory ทำให้ VMA สี่ตัวหายไปจากแผนที่ VmSize ลด 120 MiB (64 + 32 + 8 + 16) และ RSS ลดจาก 106.5 เหลือ 10.5 MiB region 16 MiB ที่เคยมี stack ของ thread 367 ปนอยู่กลับเหลือ 8 MiB เท่ากับ stack ตามเดิม ขั้นที่ 16 ให้ผลที่ละเอียดกว่าคำอธิบายในโปรแกรม demo หลัง free() block เล็กทั้งหมดและเรียก malloc_trim(0) ขนาดของ [heap] ลดลงเพียง 40 KiB (2,112 → 2,072 KiB) แต่ RSS ของ heap ลดจาก 2,068 KiB เหลือเพียง 20 KiB ตามเอกสารของ glibc ตั้งแต่เวอร์ชัน 2.8 malloc_trim() คืนหน่วยความจำของทุก chunk ที่มีหน้าว่างทั้งหน้าด้วย madvise() ไม่ได้จำกัดอยู่แค่การหด brk ที่ปลาย heap [25] heap จึงเกือบไม่หดในแง่ที่อยู่ แต่หน้าจริงเกือบทั้งหมดถูกคืนแล้ว

ไฟล์ที่ถูกลบและหน่วยความจำร่วม

region ไฟล์ในขั้นที่ 10 แสดงชื่อไฟล์ตามด้วย (deleted) เพราะ demo ลบไฟล์ทันทีหลังสร้าง แต่ mapping ยังอ้างถึง inode อยู่ ไฟล์จึงยังไม่หายจริงจนกว่าจะ munmap หน้าที่อ่านถูกนับเป็น RssFile (1.6 → 9.6 MiB) เพราะเป็น page cache ของไฟล์ ส่วน MAP_SHARED | MAP_ANONYMOUS ในขั้นที่ 11 kernel สร้างจาก shmem และแสดงใน maps เป็น /dev/zero (deleted) ที่มีสิทธิ์ rw-s ถูกนับใน RssShmem mapping แบบนี้ส่งต่อให้โปรเซสลูกหลัง fork() เพื่อใช้หน่วยความจำร่วมกันได้ [21]

2.6 ข้อจำกัด

เครื่องมือใช้ได้เฉพาะบน Linux เพราะพึ่ง /proc และดูได้เฉพาะโปรเซสของผู้ใช้เดียวกันหรือเมื่อเป็น root ถ้าอ่าน smaps ไม่ได้จะเหลือเพียงข้อมูลจาก maps ซึ่งไม่มี RSS การจำแนก region บางชนิดเป็น heuristic ดังที่เห็นในกรณี arena ที่ถูกรวมกับ stack และ region 16 MiB หลัง mprotect การระบุ thread stack อย่างแม่นยำต้องอ่าน stack pointer จาก /proc/PID/task/TID/syscall ซึ่งอ่านได้เฉพาะเมื่อมีสิทธิ์ (เช่นเป็นโปรเซสลูกที่เครื่องมือเปิดเอง) และ thread ไม่ได้กำลังทำงานอยู่บน CPU snapshot แต่ละครั้งอ่านหลายไฟล์ไม่พร้อมกัน ถ้าโปรเซสเปลี่ยน map ระหว่างอ่าน ค่าอาจไม่สอดคล้องกันเล็กน้อย และการอ่าน pagemap ของ region ขนาดใหญ่มีต้นทุน เครื่องมือจึงจำกัดไว้ที่ region ไม่เกิน 1 GiB และรวมไม่เกิน 2^{23} หน้าต่อ snapshot สุดท้าย kernel ของ WSL2 ที่ใช้ไม่ได้เปิดการตั้งชื่อ anonymous region ทำให้ VMA ถูกรวมกันบ่อยกว่าบน kernel ที่รองรับ

ส่วนที่ 3 การใช้ปัญญาประดิษฐ์ (AI)

ตามที่โจทย์กำหนดให้เปิดเผยการใช้ AI ในทั้งสองข้อ รายละเอียดมีดังนี้

โค้ด การทดลอง และกราฟ โค้ดทั้งหมดในงานนี้ (parallel.c, bench.py, plot.py, mmap.py, demo.c และ Makefile) เขียนโดย AI coding assistant ชื่อ Claude Code (Anthropic, โมเดล Claude Opus 5.5) และ AI เป็นผู้รันการทดลอง (bench.py และ plot.py) บนเครื่องของนิสิต กราฟจาก results/plots ที่ใช้ในรายงานนี้ (รูปที่ 1–3 และ 5–11) จึงสร้างโดยโค้ดที่ AI เขียน ระหว่างเขียนส่วนที่วาดกราฟใน plot.py และหน้า HTML ของ mmap.py AI ได้โหลด skill ชื่อ dataviz ที่มากับ Claude Code (สำเนาอยู่ในไฟล์ SKILL.md ของแต่ละโฟลเดอร์) นิสิตเป็นผู้ตรวจสอบโค้ดและผลลัพธ์

รายงาน นิสิตใช้ Claude (Anthropic, โมเดล Claude Opus 5.5) ผ่าน claude.ai ช่วยในการจัดทำรายงานฉบับนี้ ได้แก่ การจัดทำบรรณานุกรม การวาดกราฟ การจัดทำตาราง และการจัดเรียงเนื้อหา รวมถึงการวางโครงหรือร่างส่วนประกอบต่างๆ ในรายงาน นิสิตเป็นผู้รับผิดชอบการตรวจทานเนื้อหาฉบับสุดท้าย

สรุป

ในส่วนที่ 1 blocked เป็นอัลกอริทึมที่เร็วที่สุดและขยายตัวตามจำนวน thread ได้ดีที่สุดเมื่อข้อมูลใหญ่กว่า cache เพราะการใช้ tile ซ้ำทำให้ arithmetic intensity สูงพอที่จะเป็น compute-bound ได้ถึง 232 GFLOP/s ที่ n = 800 และ 178 GFLOP/s ที่ n = 3200 ส่วน ikj และ transpose เร็วที่สุดเมื่อข้อมูลจุใน cache (397 GFLOP/s ที่ n = 200) แต่กลายเป็น memory-bound เมื่อ B ใหญ่กว่า L3 speedup สูงสุดของทั้งโปรแกรมคือ 5.44 เท่าบน 8 thread ซึ่งถูกจำกัดทั้งโดยส่วน serial ที่มีงาน $O(n^2)$ และโดย overhead ที่เพิ่มตามจำนวนคอร์ (ความถี่ bandwidth และการแย่ง CPU กับ host ของ WSL2 ซึ่งเห็นชัดที่สุดเมื่อจำนวน thread เท่ากับจำนวน CPU พอดี) ค่าตั้งที่มีผลมากที่สุดได้แก่ flag ของ compiler (เร็วขึ้น 14 เท่าจาก -O0 ถึง -O3 -march=native) ขนาด tile (ต่างกันถึง 6 เท่า) n ที่เป็นกำลังของสอง (naive ช้าลง 5.6 เท่า) ขนาด chunk ที่ไม่เหมาะกับจำนวนงาน (2.3 เท่า) และการผูก thread ทั้งทีมไว้กับ CPU เดียวร่วมกับการ spin ของ runtime (ช้าลง 355 เท่า) ส่วน huge pages ลด page fault ได้ 12–250 เท่าอย่างที่คำนวณได้ทุกหลัก แต่ผลต่อเวลา kernel ส่วนใหญ่อยู่ในระดับ noise ส่วนขนานต่อส่วน serial อยู่ที่ประมาณ 7 : 1 ถึง 39 : 1 เมื่อนับเฉพาะโค้ด serial และ 2.7 : 1 ถึง 27 : 1 เมื่อนับ overhead ด้วย และเข้าใกล้ 100% เมื่อ n โตตามแนวคิดของ Gustafson's law

ในส่วนที่ 2 เครื่องมือ mmap.py แสดง memory map ได้สามระดับ ตั้งแต่ภาพรวมของโปรเซส region และหน้า ทั้งใน terminal และใน HTML แบบโต้ตอบที่มี timeline กรณีศึกษา demo แสดงแนวคิดหลักของ virtual memory ได้ครบด้วยตัวเลขจริง ได้แก่ demand paging การเลือก brk หรือ mmap ของ malloc การรวมและแยก VMA ของ kernel stack และ guard page ของ thread malloc arena และความต่างระหว่างการคืนช่วงที่อยู่ด้วย munmap กับการคืนหน้าจริงด้วย madvise รวมถึงข้อจำกัดของการจำแนก region จากข้อมูลระดับ VMA

เอกสารอ้างอิง

[1] OpenMP Architecture Review Board, *OpenMP Application Programming Interface, Version 5.2*, Nov. 2021. [Online]. Available: https://www.openmp.org/specifications/

[2] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conf.*, Atlantic City, NJ, USA, 1967, pp. 483–485, doi: 10.1145/1465482.1465560.

[3] AMD, "AMD Ryzen 7 9700X Desktop Processor — specifications." [Online]. Available: https://www.amd.com/en/products/processors/desktops/ryzen/9000-series/amd-ryzen-7-9700x.html

[4] B. Cohen and M. Subramony, "Next generation 'Zen 5' core," presented at *Hot Chips 36*, Stanford, CA, USA, Aug. 2024. [Online]. Available: https://www.hc2024.hotchips.org/assets/program/conference/day2/24_HC2024.AMD.Cohen.Subramony.final.pdf

[5] Microsoft, "Windows Subsystem for Linux documentation." [Online]. Available: https://learn.microsoft.com/en-us/windows/wsl/

[6] Free Software Foundation, *GNU libgomp: GNU Offloading and Multi Processing Runtime Library* (sections on OMP_PROC_BIND, OMP_PLACES, OMP_WAIT_POLICY, OMP_SCHEDULE, GOMP_SPINCOUNT). [Online]. Available: https://gcc.gnu.org/onlinedocs/libgomp/

[7] Linux kernel documentation, "Transparent Hugepage Support." [Online]. Available: https://docs.kernel.org/admin-guide/mm/transhuge.html

[8] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[9] U. Drepper, "What every programmer should know about memory," Red Hat, Inc., Nov. 2007. [Online]. Available: https://people.freebsd.org/~lstewart/articles/cpumemory.pdf

[10] M. S. Lam, E. E. Rothberg, and M. E. Wolf, "The cache performance and optimizations of blocked algorithms," in *Proc. 4th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS IV)*, 1991, pp. 63–74, doi: 10.1145/106972.106981.

[11] K. Goto and R. A. van de Geijn, "Anatomy of high-performance matrix multiplication," *ACM Trans. Math. Softw.*, vol. 34, no. 3, Art. no. 12, May 2008, doi: 10.1145/1356052.1356053.

[12] R. Freivalds, "Probabilistic machines can use less running time," in *Information Processing 77, Proc. IFIP Congress*, 1977, pp. 839–842.

[13] Linux kernel documentation, "The /proc Filesystem" (sections /proc/PID/maps, /proc/PID/smaps, /proc/PID/status). [Online]. Available: https://docs.kernel.org/filesystems/proc.html

[14] Linux man-pages project, "madvise(2)," "getrusage(2)," "taskset(1)." [Online]. Available: https://man7.org/linux/man-pages/

[15] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[16] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009, doi: 10.1145/1498765.1498785.

[17] A. H. Karp and H. P. Flatt, "Measuring parallel processor performance," *Commun. ACM*, vol. 33, no. 5, pp. 539–543, May 1990, doi: 10.1145/78607.78614.

[18] J. L. Gustafson, "Reevaluating Amdahl's law," *Commun. ACM*, vol. 31, no. 5, pp. 532–533, May 1988, doi: 10.1145/42411.42415.

[19] Linux man-pages project, "proc_pid_maps(5)" and "proc_pid_smaps(5)." [Online]. Available: https://man7.org/linux/man-pages/man5/proc_pid_maps.5.html

[20] Linux kernel documentation, "Examining Process Page Tables" (/proc/PID/pagemap). [Online]. Available: https://docs.kernel.org/admin-guide/mm/pagemap.html

[21] Linux man-pages project, "mmap(2)," "mprotect(2)," "brk(2)," "mallopt(3)" (M_MMAP_THRESHOLD), "prctl(2)" (PR_SET_VMA_ANON_NAME). [Online]. Available: https://man7.org/linux/man-pages/

[22] The GNU C Library manual, "Memory Allocation" (malloc, arenas, mmap threshold). [Online]. Available: https://www.gnu.org/software/libc/manual/

[23] M. Kerrisk, *The Linux Programming Interface*. San Francisco, CA, USA: No Starch Press, 2010.

[24] M. Gorman, *Understanding the Linux Virtual Memory Manager*. Upper Saddle River, NJ, USA: Prentice Hall, 2004.

[25] Linux man-pages project, "malloc_trim(3)." [Online]. Available: https://man7.org/linux/man-pages/man3/malloc_trim.3.html